esprit

Se former autrement

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

CANvisionNative

By: Mohamed Mootaz Ben Kraiem

Academic supervisor: Nadia Boulifa

Corporate Internship Supervisor: Hamda Guizani

Capgemini engineering

cdio

EUR-ACE

Cti

CONFÉRENCE DES GRANDES ÉCOLES

Acknowledgements

I would like to express my sincere gratitude to my family to both my parents for their continuous support throughout this project and throughout my academic journey. I am especially grateful to my mother for her unconditional love, endless encouragement, and for every sacrifice she has made to help me reach this point. None of this would have been possible without her.

I would also like to thank my brothers and my entire family for always believing in me and supporting me through every challenge.

Finally, I would like to express my deepest appreciation to the person who stood by my side every step of the way. To everyone who supported me, directly or indirectly, thank you. This accomplishment is as much yours as it is mine.

Abstract

Modern vehicles, and electric vehicles in particular, rely on the Controller Area Network (Controller Area Network (CAN)) bus to let their electronic control units exchange data. The CAN protocol was designed for reliability on a closed bus, not for security: it carries no built-in authentication or encryption. In a Battery Management System (BMS), where CAN traffic reflects safety-critical values such as cell voltage, state of charge, and temperature, engineers need visibility into this traffic that goes beyond passively logging frames.

This report presents **CANvisionNative**, an intelligent CAN diagnostic and validation platform for Battery Management Systems that incorporates machine-learning-based anomaly detection. The system combines a Windows desktop client, built with Windows Presentation Foundation (WPF) and Model-View-ViewModel (MVVM), with a Python backend exposing a Representational State Transfer (REST) Application Programming Interface (API) built on FastAPI. The backend decodes raw CAN frames using vehicle-specific CAN Database (signal definition file format) (DBC) files, extracts timing and payload features, and scores each frame with a fusion of four analysis layers: a timing layer, a payload-continuity layer, a per-vehicle Isolation Forest machine-learning layer, and a temporal-persistence layer. Detected anomalies are explained in natural language through an AI explanation pipeline that combines rule-based reasoning with a Large Language Model (Large Language Model (LLM)) provider chain.

The platform supports three operating modes: replay of recorded logs (Comma-Separated Values (CSV), Measurement Data Format version 4 (ASAM MDF4) (MF4), candump, Vector CAN), live simulation, and live hardware capture through an ELM327 or SocketCAN adapter. Its machine-learning layer was trained on more than seven million CAN frames from sixteen vehicle- and dataset-specific sources and validated on a real one-hour Kia EV6 recording containing 97,741 frames.

The project followed an evidence-based engineering methodology: every claim about system behaviour was checked against reproducible, measured evidence before being accepted. This discipline produced a fully corrected detection pipeline, whose final release reduces alert volume by 25% and eliminates machine-learning score saturation entirely, without loosening the detection threshold.

This report documents the requirements, architecture, implementation, and validation of CANvisionNative, and shows that a desktop-class, explainable, multi-layer CAN diagnostic and validation platform can be built and rigorously verified for real vehicle CAN bus data using commodity hardware and open datasets.

Keywords: Controller Area Network, Battery Management System, CAN Diagnostics, Electric

Vehicle, Machine Learning, Isolation Forest, Anomaly Detection, Large Language Model, Automotive Validation Engineering.

Résumé

Les véhicules modernes, en particulier les véhicules électriques, reposent sur le bus CAN (Controller Area Network) pour permettre à leurs calculateurs (Electronic Control Unit (ECU)) d'échanger des données. Le protocole CAN a été conçu pour la fiabilité sur un bus fermé, et non pour la sécurité : il ne dispose d'aucune authentification ni chiffrement natif. Dans un système de gestion de batterie (BMS), où le trafic CAN reflète des valeurs critiques pour la sécurité telles que la tension des cellules, l'état de charge et la température, les ingénieurs ont besoin d'une visibilité sur ce trafic qui va au-delà d'un simple enregistrement passif des trames.

Ce rapport présente **CANvisionNative**, une plateforme intelligente de diagnostic et d'aide à la validation CAN pour les systèmes de gestion de batterie (BMS), intégrant une détection d'anomalies fondée sur l'apprentissage automatique. Le système combine un client de bureau Windows, développé en WPF selon le patron MVVM, avec un backend Python exposant une API REST construite avec FastAPI. Le backend décode les trames CAN brutes à l'aide de fichiers DBC spécifiques à chaque véhicule, extrait des caractéristiques temporelles et de contenu, puis évalue chaque trame par la fusion de quatre couches d'analyse : une couche temporelle, une couche de continuité de charge utile, une couche d'apprentissage automatique (Isolation Forest) propre à chaque véhicule, et une couche de persistance temporelle. Les anomalies détectées sont expliquées en langage naturel grâce à un module d'explication combinant un raisonnement à base de règles et une chaîne de modèles de langage (LLM).

La plateforme prend en charge trois modes de fonctionnement : le rejeu de journaux enregistrés (CSV, MF4, candump, Vector CAN), la simulation en direct, et la capture matérielle en direct via un adaptateur ELM327 ou SocketCAN. Sa couche d'apprentissage automatique a été entraînée sur plus de sept millions de trames CAN provenant de seize sources de véhicules et de jeux de données, et validée sur un enregistrement réel d'une heure d'une Kia EV6 contenant 97 741 trames.

Le projet a suivi une méthodologie d'ingénierie fondée sur la preuve : chaque affirmation sur le comportement du système a été vérifiée par des mesures reproductibles avant d'être retenue. Cette rigueur a permis d'aboutir à un pipeline de détection entièrement corrigé, dont la version finale réduit le volume d'alertes de 25 % et élimine totalement la saturation du score d'apprentissage automatique, sans assouplir le seuil de détection.

Ce rapport documente les besoins, l'architecture, l'implémentation et la validation de CANvisionNative, et montre qu'une plateforme de diagnostic et d'aide à la validation CAN, multicouche, explicable et de classe bureautique, peut être construite et rigoureusement vérifiée pour des données réelles de bus CAN de véhicule, en utilisant du matériel courant et des jeux de données

ouverts.

Mots-clés : Bus CAN, Système de gestion de batterie, Diagnostic CAN, Véhicule électrique, Apprentissage automatique, Isolation Forest, Détection d'anomalies, Modèle de langage, Ingénierie de validation automobile.

Contents

List of Figures

List of Tables

General Introduction

Vehicles have become distributed computer networks on wheels. A modern car, and even more so an electric vehicle, contains dozens of ECUs that control braking, battery management, motor torque, infotainment, and driver-assistance functions. These ECUs talk to each other over shared internal networks, and the most common of these networks, by far, is the CAN bus.

The CAN protocol was standardized in the 1980s for industrial reliability: every node on the bus can send and receive frames, and the protocol guarantees that the highest-priority message wins in case of collision. This design works extremely well for a closed, trusted environment. It was never designed to resist an attacker who gains access to the bus, because it has no field for authentication and no encryption. Once an attacker can inject frames onto the bus, the bus itself cannot tell a legitimate frame from a forged one.

Over the last fifteen years, security researchers have repeatedly shown that this is not a theoretical problem. Remote attacks against production vehicles, published diagnostic exploits, and public intrusion-detection datasets have all demonstrated that a compromised CAN bus can affect physical vehicle behaviour. At the same time, the rise of electric vehicles has increased the number of safety-critical messages on the bus (battery management, motor control) and the number of remote attack surfaces (telematics, over-the-air updates, mobile apps).

An intrusion detection system for the CAN bus has to work under constraints that are different from a typical network Intrusion Detection System (IDS): frames are small (up to 8 bytes of payload), periodic, and vehicle-specific, and a single trained model rarely generalizes across manufacturers. This motivates a system that combines several complementary detection strategies (timing, payload, machine learning, temporal persistence) and that adapts to the specific vehicle being monitored.

This report describes the design, implementation, and validation of **CANvisionNative**, an intelligent CAN diagnostic and validation platform for Battery Management Systems that incorporates machine-learning-based anomaly detection. The platform is built to detect anomalies on recorded and live CAN traffic, to explain detected anomalies in plain language, and to export the results for later analysis. The project also includes a full engineering validation phase, in which the system was tested against a real one-hour vehicle recording, defects were identified with concrete evidence, fixed, and re-validated.

Report structure

This report is organized into five chapters.

Chapter 1 presents the general framework of the project: the host organization, the project context, the problem statement, the objectives, and the development methodology followed. Chapter 2 presents the requirements analysis: a short review of existing solutions, the functional and non-functional requirements, the actors, and the use cases. Chapter 3 presents the system design: the technology choices and the overall architecture, illustrated with UML diagrams. Chapter 4 describes the implementation of the main modules, illustrated with real screenshots of the running application. Chapter 5 presents the validation and results: the testing methodology, the real-data validation on the Kia EV6 recording, the six defects found and fixed, the before/after measurements, and an assessment of the system's software readiness. The report ends with a general conclusion, a bibliography, and a set of appendices.

Chapter 1

General Framework

1.1 Introduction

This chapter sets the general framework of the project. It presents the host organization, the context that led to the project, the problem it addresses, the objectives that guided the work, and the development methodology followed. It closes with a short summary that leads into the requirements analysis of Chapter 2.

1.2 Host organization

1.2.1 Presentation

This project was carried out at **Capgemini**, a global technology and consulting company headquartered in France. Capgemini provides IT consulting, digital transformation, and engineering services to clients across several industries, including automotive and other transportation sectors, where it works on both software systems and embedded/engineering projects. As a large, multi-industry technology group, Capgemini regularly hosts end-of-studies internship projects that combine academic engineering training with a concrete technical deliverable.

1.2.2 Department and internship context

The internship was carried out within Capgemini Tunisia's Automotive division, in the Battery Management Systems (BMS) team. This team works on embedded software and validation for battery management functions in electric and hybrid vehicles, which places it directly next to the kind of in-vehicle network traffic this project analyses. Although CANvisionNative is not itself a BMS deliverable, its role as a CAN diagnostic and validation platform placed it directly alongside the kind of in-vehicle network traffic the BMS team analyses day to day, and the internship benefited from being carried out in a team already familiar with automotive ECU communication, CAN bus behaviour, and the general constraints of embedded automotive software.

The project was carried out as a final-year engineering internship (Projet de Fin d'Études (Final Year Engineering Project) (PFE)). It was framed as an individually driven technical project: the student was responsible for the full life cycle of CANvisionNative, from requirements to design, implementation, and validation, with periodic guidance from an academic supervisor at ESPRIT.

1.3 Project context

The project sits in the broader context of automotive cybersecurity. Modern vehicles, and electric vehicles in particular, are built around internal networks of ECUs that communicate over a CAN bus. This bus was designed for reliability, not for security, and it has been repeatedly shown in the security literature to be vulnerable to message injection, flooding, and replay attacks (Chapter 2 reviews this in more detail).

The project, **CANvisionNative**, is an intelligent CAN diagnostic and validation platform for Battery Management Systems that incorporates machine-learning-based anomaly detection. It combines:

- a Windows desktop client (WPF, MVVM) for visualization, replay control, and analyst interaction;
- a Python backend (FastAPI) that decodes CAN frames, extracts features, scores anomalies, and generates natural-language explanations;
- a machine-learning layer trained on more than seven million CAN frames across sixteen vehicle- and dataset-specific sources;
- support for three operating modes: replay of recorded logs, live simulation, and live hardware capture.

The system was designed to be usable both as a forensic tool — importing a recorded log and reviewing detected anomalies after the fact — and, once hardware is connected, as a live monitoring tool.

1.4 Problem statement

Three problems motivated this project.

First, the CAN bus, standardized as ISO 11898 [1], has no built-in security. Any device connected to the bus can send frames, and no frame carries a sender identity or a signature. Continuous, anomaly-based monitoring is therefore one of the only practical ways to add a security-relevant layer of visibility to an existing vehicle without redesigning its electrical architecture.

Second, a single detection strategy is not enough. Timing-based detection catches flooding attacks but misses slow, low-rate attacks. Payload-based detection catches out-of-range values but

misses attacks that stay within plausible ranges. Machine-learning models trained on one vehicle rarely transfer cleanly to another vehicle, because message layout, frequency, and normal-value ranges are vehicle-specific. This suggested a design combining several independent detection layers, fused into a single score, rather than relying on one algorithm.

Third, a raw anomaly score is not enough for a human analyst to act on. A list of alerts with numeric scores does not explain what happened, why the system flagged it, or what to do next. This motivated an explanation layer that turns a detected anomaly into a short, readable description of the likely attack pattern and a recommended action.

Beyond building the system, this project also required a rigorous validation phase, since a detection system that produces incorrect or misleading alerts is not trustworthy. This is documented in full in Chapter 5, which describes six confirmed software defects that were found through evidence-based testing on a real one-hour vehicle recording, and their correction.

1.5 Objectives

1.5.1 General objective

The general objective of this project is to design, implement, and rigorously validate a multi-layer CAN diagnostic and validation platform for Battery Management Systems, incorporating machine-learning-based anomaly detection, that adapts to the specific vehicle being analysed and explains its own alerts in plain language.

1.5.2 Specific objectives

1. Design and implement a multi-layer CAN anomaly-detection pipeline combining timing, payload, machine-learning, and temporal-persistence signals into a single fusion score.

2. Support multiple input sources: recorded logs (CSV, MF4, candump, Vector CAN), a live attack simulator, and live hardware capture through an ELM327 or SocketCAN adapter.

3. Train and integrate vehicle-specific machine-learning models so that detection adapts to the vehicle being analysed instead of relying on one generic model.

4. Build a desktop client that lets an analyst import a log, replay it, review detected anomalies, and export a report.

5. Add a natural-language explanation layer that describes each anomaly and recommends an action, using a combination of rule-based reasoning and a Large Language Model.

6. Validate the complete pipeline against a real vehicle recording, using measurable, reproducible evidence, and correct any defect found before considering the system ready.

1.6 Development methodology

1.6.1 Waterfall, Agile, and Scrum

Table 1.1 compares the three methodological approaches usually considered for a project of this size, before explaining which one guided this project and why.

Table 1.1: Comparison of Waterfall, Agile, and Scrum.

Approach	**Advantages**	**Disadvantages**
Waterfall	Clear, predictable phases; easy to plan and document up front; well suited to fixed, well-understood requirements.	Requirements are frozen too early for an exploratory project; a defect found late (for example during validation) is expensive to trace back to an earlier phase; poor fit when the exact scope of the machine-learning layer or the validation effort cannot be fully specified in advance.
Agile (general)	Iterative delivery of working software; requirements and design can evolve as understanding improves; feedback is incorporated continuously instead of only at the end.	Requires discipline to avoid scope drift; without a team, several Agile ceremonies (stand-ups, planning poker) lose most of their value.
Scrum	Structures Agile delivery around a prioritized backlog and short, incremental delivery cycles, each producing a working increment that can be reviewed and built upon; makes it natural to reprioritize (for example, to insert a dedicated correction phase) once real evidence changes what matters most.	Formal Scrum assumes a cross-functional team with distinct roles (Product Owner, Scrum Master, development team) and regular ceremonies; a single-developer project cannot reproduce these roles literally and has to adapt the framework rather than apply it as written.

1.6.2 Why Scrum was chosen for this project

Scrum was chosen, in an adapted form, over a strict Waterfall plan because the project's two main sources of uncertainty — how well a vehicle-specific machine-learning model would actually perform, and how many real defects a rigorous validation pass would uncover — could not be estimated accurately before the work started. A Waterfall plan would have required freezing the detection design and the validation scope before either was known with any confidence.

An incremental approach, where each milestone produced a working, testable piece of the system before the next one was scoped in detail, matched the way the project's own uncertainty was actually resolved: one milestone at a time, backed by evidence rather than by an up-front estimate.

It should be stated plainly that this was a single-person project, not a Scrum team. There was no Scrum Master, no separate Product Owner, and no fixed-length sprint calendar with daily stand-up meetings in the literal Scrum sense. What was kept from Scrum was its underlying discipline: a prioritized backlog, incremental delivery of working functionality, a review step before moving on, and a clear definition of done. Figure 1.1 shows the standard Scrum cycle used here as a reference model; the rest of this section explains, phase by phase, how that discipline was actually applied on this project, without claiming that every ceremony in the diagram was run in its formal, textbook form.

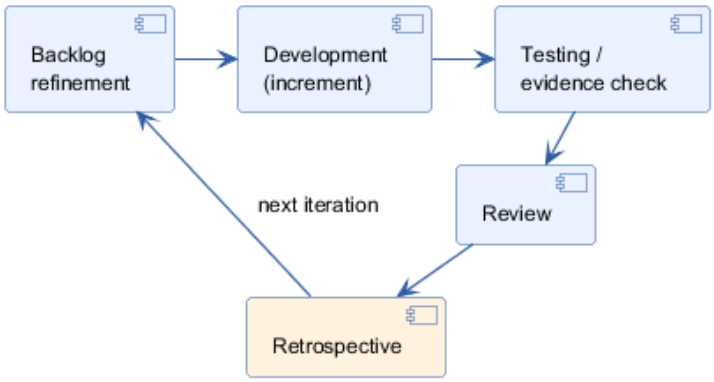

Figure 1.1: Reference Scrum cycle used as the framework for this project's iterative process.

1.6.3 How the real process reflected Scrum concepts

Product backlog. The project's internal roadmap (`ROADMAP.md`) served as a living backlog, listing milestones M1 through M8 (live foundation, hardware layer, multi-state Machine Learning (ML) training, Artificial Intelligence (AI) explanation engine, real-time graphs, session recording, integration, offline/live decode pipeline) in priority order, re-ordered as earlier milestones revealed what the later ones actually needed.

Incremental development. Each milestone was scoped to deliver a working, demonstrable increment of the system — for example, M1 delivered a functioning live pipeline before any machine-learning model existed, and M4 added the AI explanation layer only once the detection pipeline it explains was already producing alerts. This mirrors the Scrum principle of shippable increments more closely than a single end-to-end build attempted in one pass.

Engineering correction phase as a stabilization iteration. Once the milestone-based increments were functionally complete, the project entered a dedicated, evidence-based correction phase (Chapter 5), comparable to a Scrum stabilization or hardening iteration: no new features were added, and the entire effort was redirected to finding and fixing defects, with each fix re-verified before moving to the next.

Validation iterations as reviews. Where a Scrum team would hold a sprint review with

stakeholders, this project substituted a stricter, evidence-based review: every claim about the system's behaviour had to be demonstrated on real data before it was accepted, and every fix was checked against reproducible evidence rather than presented for a subjective sign-off (Table 5.1 in Chapter 5).

Repository freeze and Release Candidate. Once the correction phase closed with all identified defects fixed and re-verified end to end, the repository was frozen for engineering changes, and the resulting state was treated as a Release Candidate — the practical equivalent of a Scrum *definition of done* applied to the whole project rather than to a single sprint.

Figure 1.2 places all of this on a single timeline: the milestone-based increments (M1, M3, M4, M5, M6, then M8/M7 together, then the code-complete-but-hardware-blocked M2), followed by the dedicated engineering correction phase, the validation re-run, and the resulting Release Candidate.

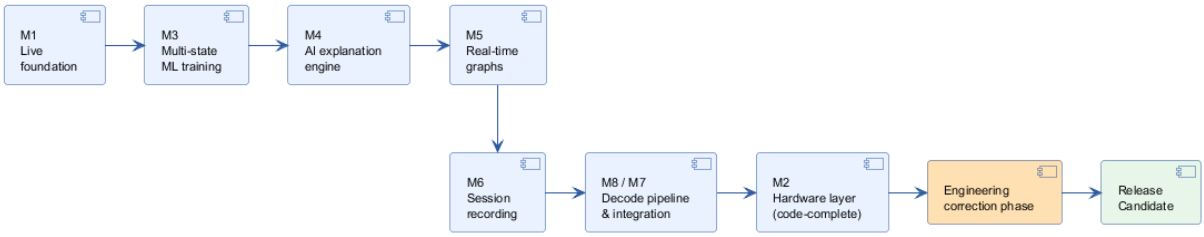

Figure 1.2: Project timeline: milestone-based increments through to the Release Candidate.

1.6.4 Conclusion

Framed this way, the project followed an incremental, evidence-driven process consistent with Scrum's core principles, adapted honestly to the reality of a single-developer internship rather than dressed up with ceremonies that were never actually run.

1.7 Conclusion

This chapter presented the host organization, the context and motivation behind CANvisionNative, the problems it addresses, the objectives set at the start of the project, and the development methodology followed. The next chapter turns this context into a concrete requirements analysis.

Chapter 2

Requirements Analysis

2.1 Introduction

This chapter turns the context and objectives from Chapter 1 into a concrete requirements analysis. It starts with a short review of existing solutions, to position what CANvisionNative adds. It then lists the functional and non-functional requirements, identifies the actors, and presents the use cases with a UML use-case diagram and textual descriptions.

2.2 Existing solutions analysis

2.2.1 Traditional CAN tools

The most common way to observe CAN traffic today is a passive sniffing and logging tool: open-source utilities such as `candump` and SavvyCAN, or commercial tools such as Vector CANoe/CANalyzer. These tools capture and display raw frames, decode them against a DBC file if one is provided, and let an engineer replay a recorded log. They are built for development and diagnostics, not for security monitoring: they show what is on the bus, but they do not judge whether a frame is normal or malicious.

2.2.2 Vehicle diagnostics tools

A second category of tools targets vehicle diagnostics rather than raw CAN traffic: On-Board Diagnostics, second generation (OBD-II) (SAE J1979 [2]) scanners and Unified Diagnostic Services (UDS) (ISO 14229 [3]) diagnostic clients query specific ECUs for known parameters (fault codes, sensor values) through a request/response protocol. These tools are useful for maintenance and repair, but they only see what they explicitly ask for, and they have no notion of monitoring the bus continuously for unexpected behaviour.

2.2.3 Existing intrusion detection approaches

Published intrusion detection approaches for the CAN bus generally fall into three families. **Signature-based detection** matches traffic against known attack patterns; it is precise for known attacks but cannot catch a new one. **Specification-based detection** encodes expected behaviour by hand (expected identifiers, frequency ranges, payload ranges); it does not need attack data, but it needs a manually written specification for every vehicle and does not generalize. **Anomaly-based detection** learns what normal traffic looks like from timing and/or payload statistics and flags deviations; it is the most broadly applicable of the three, because it only needs a sample of normal traffic, and it is the detection family the anomaly-detection layer of CANvisionNative belongs to.

2.2.4 Current limitations

Across the tools and approaches above, three limitations recur. First, most anomaly-based designs rely on a single detection signal (either timing or one machine-learning model), so an attack that does not disturb that one signal goes undetected. Second, a machine-learning model trained on one vehicle is rarely reused correctly across vehicles, because message layout, frequency, and normal-value ranges are vehicle-specific; in practice, systems that mention vehicle-specific models often treat correct model selection as an implementation detail rather than something to verify explicitly. Third, none of the tools reviewed above turn a raw anomaly score into a plain-language explanation an analyst can act on directly.

2.2.5 Why CANvisionNative is different

CANvisionNative addresses these three limitations directly. It fuses four independent detection signals (timing, payload, machine learning, temporal persistence) into one score, so an attack missed by one layer can still be caught by another. It selects a trained model per vehicle profile at run time, rather than a single generic model; this exact selection step was one of the six defects verified during validation (Chapter 5). And it adds a natural-language explanation layer, combining rule-based reasoning with a Large Language Model, so an alert comes with a description of the likely attack pattern and a recommended action rather than a bare score.

In practice, this positions CANvisionNative between the two categories reviewed above. It monitors traffic continuously and learns what normal behaviour looks like, the way the anomaly-detection literature does, but it is packaged as a diagnostic and validation platform an analyst can actually use during a validation campaign, with an import-replay-review-export workflow, rather than as a bare research detector that only outputs a score.

2.3 Functional requirements

- **FR1 — Import a recorded log.** The system shall let the user import a CAN log file in CSV, MF4, Vector CAN (`.asc`), candump (`.log`), or PCAN text (`.txt`) format, and convert it to a canonical frame stream (timestamp, identifier, payload).

- **FR2 — Select a vehicle profile.** The system shall let the user select the vehicle that produced the log (or the closest matching profile), so that the correct DBC file and the correct trained model are used.
- **FR3 — Replay the log.** The system shall replay the imported frames through the detection pipeline, in order, at a user-selectable speed (0.5× up to a maximum speed), and report progress.
- **FR4 — Detect anomalies.** For every frame, the system shall compute a fusion anomaly score from four detection layers (timing, payload, machine learning, temporal persistence) and raise an alert when the score exceeds the configured threshold.
- **FR5 — Classify severity.** Each alert shall be labelled with a severity level (LOW, MEDIUM, HIGH, CRITICAL) derived from the fusion score.
- **FR6 — Explain an alert.** The system shall let the user request a natural-language explanation of any alert, including the dominant detection layer, a plain-language description of the likely attack pattern, and a recommended action.
- **FR7 — Export results.** The system shall let the user export the alert list as a CSV file and as a formatted PDF report.
- **FR8 — Live simulation.** The system shall be able to generate a simulated live CAN stream, including a simulated attack burst, for testing the detection pipeline without a physical vehicle.
- **FR9 — Live hardware capture.** The system shall be able to connect to a physical CAN adapter (ELM327 or SocketCAN) and feed real vehicle frames into the same detection pipeline used for replay.
- **FR10 — Record a session.** The system shall be able to record a live or simulated session to a log file, optionally injecting a labelled attack, for later replay or retraining.
- **FR11 — Configure the system.** The system shall let the user configure the alert threshold, the LLM provider key, and the hardware interface settings, and persist this configuration between sessions.

2.4 Non-functional requirements

- **NFR1 — Performance.** The replay engine shall process a one-hour, 97,741-frame recording in a few minutes, not hours, on a standard desktop machine. Chapter 5 reports a measured throughput of about 286 frames per second after the corrections described there.
- **NFR2 — Reliability.** A detection defect must be provable and fixable with concrete evidence, not by blindly tuning thresholds. This principle governed the whole validation phase in Chapter 5.

- **NFR3 — Usability.** An analyst without machine-learning expertise shall be able to import a log, read the alert list, and understand the explanation of an alert without consulting external documentation.
- **NFR4 — Portability of input formats.** Every supported log format shall enter the detection pipeline through the same canonical frame representation, so that detection behaviour does not depend on which format was used to record the data.
- **NFR5 — Extensibility.** Adding a new vehicle profile shall only require adding a DBC file and a trained model directory, without changing the detection engine code.
- **NFR6 — Graceful degradation.** If the LLM provider is unavailable, the system shall still produce a rule-based explanation instead of failing the request.
- **NFR7 — Platform.** The desktop client shall run on Windows 10/11 using .NET Framework 4.8 and WPF; the backend shall run as a local Python process exposing a REST API on `localhost`. The full software and hardware requirements are listed in Table B.1 and Table B.2 (Appendix B).

2.5 Actors

- **Analyst (primary user).** Imports logs, starts replays, reviews alerts, requests explanations, exports reports, configures the system.
- **Vehicle / CAN bus (external system).** Source of live frames when the hardware capture mode is used.
- **LLM provider (external system).** Supplies natural-language text for the explanation layer (Gemini, or a local Ollama model as fallback).
- **CANvisionNative backend (system under design).** Performs decoding, feature extraction, scoring, and explanation generation, and exposes them through the REST API.

2.6 Use cases

Figure 2.1 shows the use-case diagram for the Analyst actor, the two external systems, and the main use cases implemented by the system.

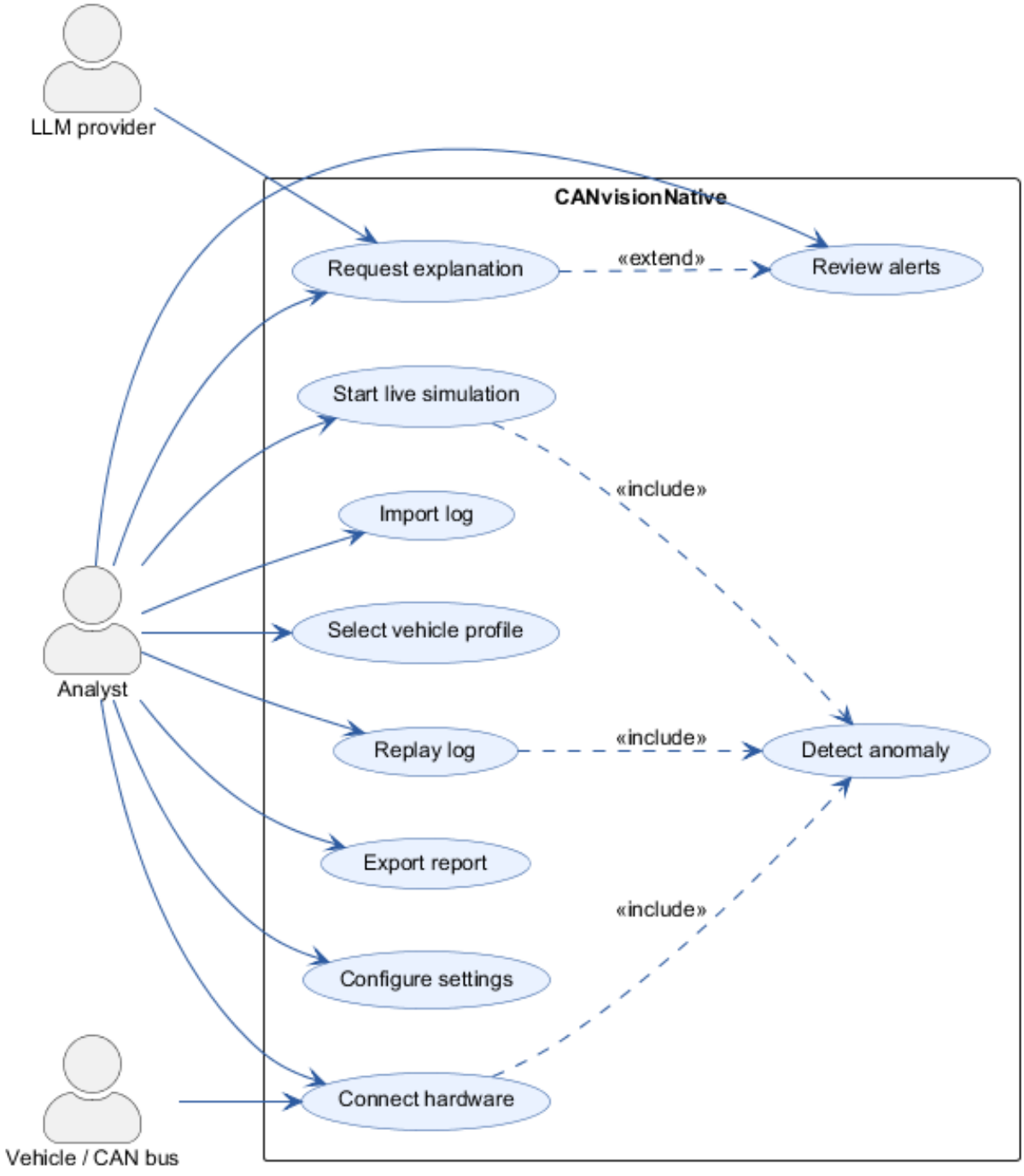

Figure 2.1: Use-case diagram for CANvisionNative.

Import log, Select vehicle profile, Review alerts, Export report, Configure settings, Start live simulation, and Connect hardware are all initiated directly by the Analyst. Detect anomaly is included by Replay log, Start live simulation, and Connect hardware, since all three input modes feed the same detection pipeline. Request explanation extends Review alerts, since it is an optional action available on any reviewed alert, and it involves the external LLM provider. Table 2.1 gives a short textual description of the eight use cases most representative of the system's behaviour.

Table 2.1: Textual description of the main use cases.

Use case	Description
Import log	The analyst selects a log file; the system detects its format and converts it to the canonical frame stream (FR1).
Select vehicle profile	The analyst chooses the vehicle that produced the log, so the correct DBC file and trained model are used (FR2).
Replay log	The system replays the canonical frame stream through the detection pipeline at a user-selectable speed, including the Detect anomaly use case (FR3, FR4).
Detect anomaly	For every frame, the system computes the four-layer fusion score and raises an alert above the configured threshold (FR4, FR5). Shared by replay, live simulation, and hardware capture.
Review alerts	The analyst browses the alert list, filters by severity, and inspects the layer-attribution breakdown of a selected alert.
Request explanation	The analyst asks for a natural-language explanation of a selected alert; the system builds context and queries the LLM provider chain, falling back to a rule-based template if needed (FR6).
Export report	The analyst exports the current alert list as CSV or as a formatted PDF report (FR7).
Configure settings	The analyst sets the alert threshold, the LLM provider key, and the hardware interface parameters, persisted between sessions (FR11).

2.6.1 Activity diagram — import and replay

Figure 2.2 shows the activity flow from importing a log to publishing its alerts to the client, corresponding to the Import log, Select vehicle profile, Replay log, and Detect anomaly use cases together.

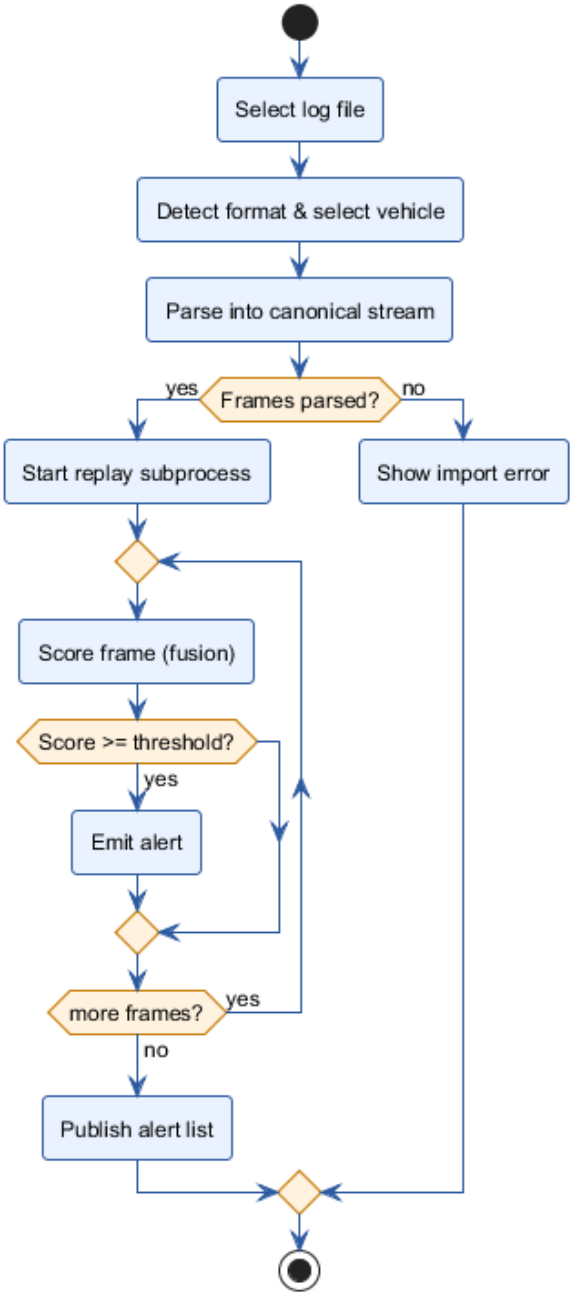

Figure 2.2: Activity diagram: import and replay a log.

2.6.2 Sequence diagram — request an AI explanation

Figure 2.3 shows the sequence of calls triggered by the Request explanation use case, including the fallback behaviour when the primary LLM provider is unavailable.

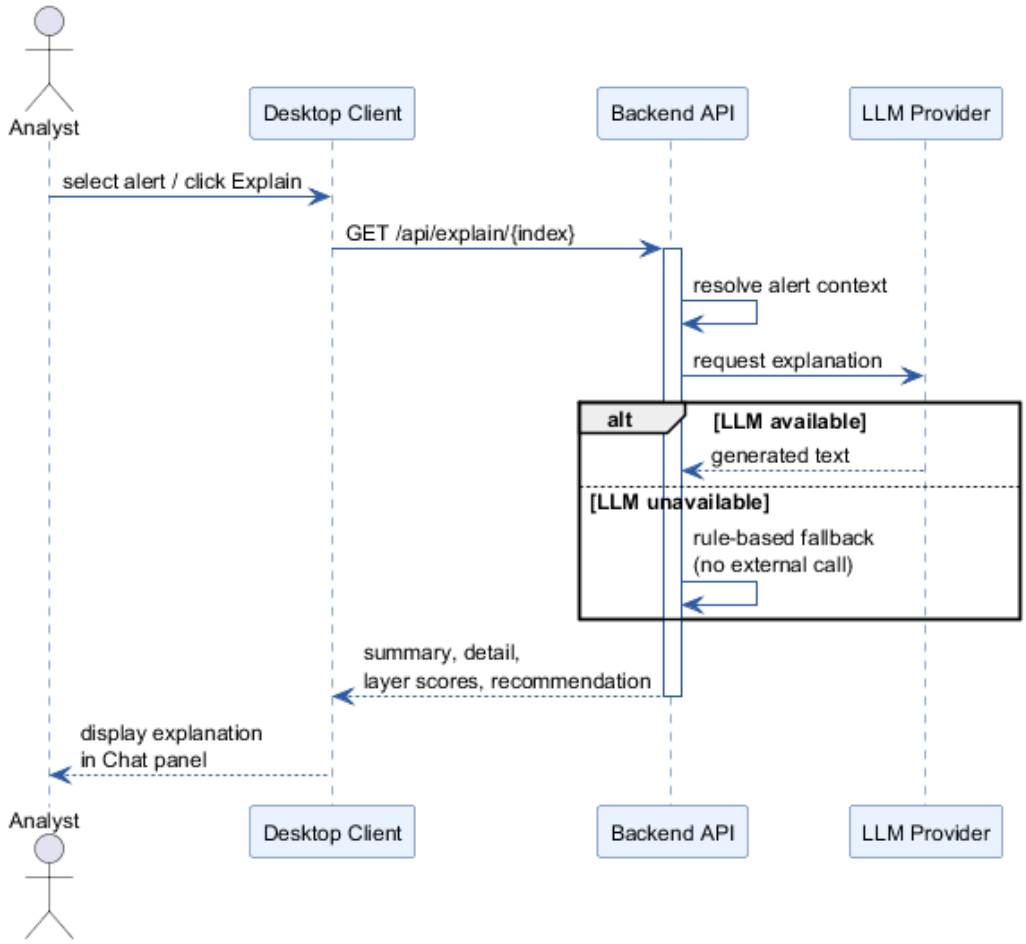

Figure 2.3: Sequence diagram: requesting an AI explanation for an alert.

2.7 Risk analysis

Table 2.2 lists the main risks identified during the project and how each was mitigated.

Table 2.2: Risk analysis.

Risk	**Likelihood**	**Impact**	**Mitigation**
ELM327 / hardware adapter not available in time	High (materialized)	Medium	Design and implement the hardware layer against the same interface as replay and simulation, so it can be validated later without changing the detection pipeline (Chapter 3).
LLM provider unavailable or unauthenticated	Medium	Low	Provider chain with automatic fallback: Gemini → local Ollama model → rule-based template.
Vehicle-specific model not selected correctly	Medium (materialized)	High	Root-cause investigation with direct evidence rather than threshold tuning; full fix and validation documented in Chapter 5.
False positives eroding analyst trust	Medium	High	Multi-layer fusion with a temporal-persistence term instead of a single-frame score; before/after alert-rate measurement in Chapter 5.
Different log formats behaving inconsistently	Medium (materialized)	Medium	Single canonical parsing entry point shared by every input path; verified in Chapter 5.
Explanation shown for the wrong alert (stale cache)	Low (materialized)	High	Root-caused to two cache-lifecycle gaps and fixed at both the completion and the start of every replay; verified with a seeded cross-dataset test (Chapter 5).

2.8 Planning

The project was carried out through a sequence of milestones, tracked internally as M1 to M8, followed by a dedicated final validation phase, as introduced in the development methodology of Chapter 1. Figure 2.4 presents this planning as a Gantt chart.

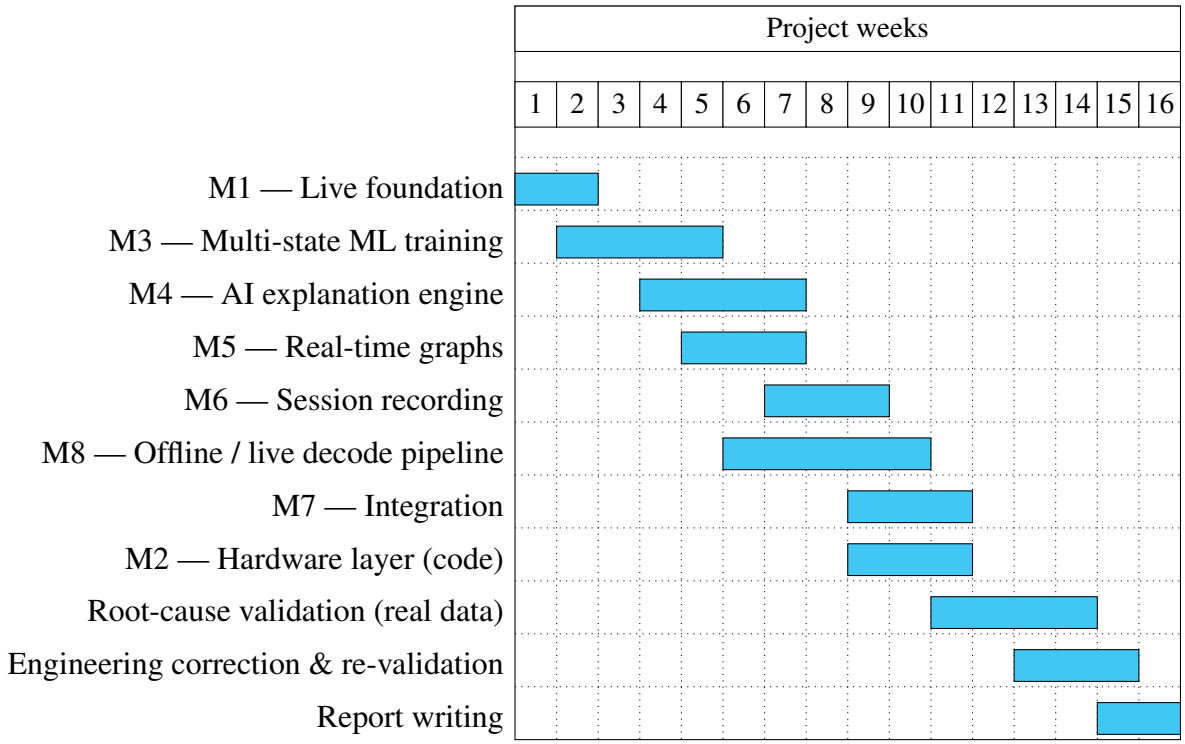

Figure 2.4: Project planning (Gantt chart, in weeks).

Milestone M2 (hardware / OBD2 validation) is shown as code-complete but is explicitly marked as blocked on the physical arrival of an ELM327 adapter, and is not part of the validated scope in Chapter 5, which is why it is not part of the risk-mitigated critical path.

2.9 Requirements traceability

Table 2.3 traces each functional requirement to the design and implementation section that realizes it and, where applicable, the validation evidence that confirms it works as intended.

Table 2.3: Requirements traceability: from requirement to implementation to validation.

Req.	Implemented in	Validated by
FR1	Canonical parser, Section 4.5.2	Fix 4, Chapter 5
FR2	Vehicle profile system, Section 4.5.4	Fix 1, Chapter 5
FR3	Replay engine, Section 4.5.1	Before/after throughput, Table 5.4
FR4	Detection pipeline, Section 3.7	Root-cause analysis, Table 5.3
FR5	Fusion scoring, Section 3.7	Severity distribution, Figure 5.7
FR6	AI explanation module, Section 3.9	Fix 6, Section 5.5.6
FR7	Export, Section on Export, Chapter 4	Export validation, Chapter 5
FR8	Live simulation (hardware layer design)	Interactive UI audit, Chapter 5
FR9	Hardware layer, Section on Hardware, Chapter 4	Code-complete; physical test pending (Limitations)
FR10	Session recording (`backend/hardware/`)	Interactive UI audit, Chapter 5
FR11	Settings, Section on Settings, Chapter 4	Interactive UI audit, Chapter 5

Every functional requirement is either backed by direct, reproducible validation evidence or explicitly marked as pending a physical test, consistent with the honest treatment of the hardware-capture limitation throughout this report.

2.10 Conclusion

This chapter reviewed existing CAN tools and intrusion-detection approaches, positioned CANvisionNative against their limitations, and defined the functional and non-functional requirements, actors, and use cases that follow from this analysis, together with the project's risk analysis and planning. The next chapter presents the system design built to satisfy these requirements.

Chapter 3

System Design

3.1 Introduction

This chapter presents the architecture of CANvisionNative: the technology choices, the overall layered architecture, the desktop client, the backend, the database, the REST API, the replay engine, the detection pipeline, the AI explanation engine, the hardware layer, and the deployment architecture. Every diagram in this chapter is a UML diagram produced with PlantUML, generated directly from the actual architecture described in Chapter 4.

3.2 Technology choices

The desktop client is built with WPF and the MVVM pattern on .NET Framework 4.8, chosen because it gives a native, responsive Windows UI with strong data-binding support, well suited to the chart-heavy, multi-screen client this project required. The backend is a Python FastAPI application, chosen because it exposes a typed, self-documenting REST API with minimal boilerplate and integrates directly with the Python machine-learning stack (scikit-learn, pandas) already used for model training. SQLite was chosen for persistence because the system runs as a single local process on a single workstation, with no need for a networked database server. Communication between the two processes uses Hypertext Transfer Protocol (HTTP) JavaScript Object Notation (JSON) over `localhost`, which keeps the client and backend independently testable and lets the backend be driven directly for automated validation, as done throughout Chapter 5.

Table 3.1 summarizes the main alternative considered for each layer and why it was set aside in favour of the chosen technology.

Table 3.1: Technology choices and the main alternative considered.

Layer	Chosen	Alternative considered	Why the alternative was set aside
Desktop client	WPF / MVVM (.NET 4.8)	Web-based client (Electron / browser)	Would add a packaging and update-delivery layer for no real benefit on a single-workstation tool, and .NET 4.8 was already the required target platform.
Backend API	FastAPI (Python)	Flask, Django	FastAPI's typed request/response models and automatic API documentation reduced boilerplate, and its async support was not needed but did not cost anything either; Django's full-stack features (templating, admin site) were unnecessary for a pure API backend.
Persistence	SQLite	PostgreSQL / MySQL	A networked database server would add an operational dependency with no benefit, since the system runs as a single local process on a single workstation.
ML runtime	scikit-learn	PyTorch / TensorFlow	The selected algorithms (Isolation Forest, K-Means) are classical, non-neural methods well supported by scikit-learn; a deep-learning framework would add dependency weight without a corresponding algorithm need (Section 4.6.2).
Client/backend transport	HTTP JSON over localhost	gRPC, named pipes	HTTP JSON is simple to test directly (curl, browser, automated scripts), which was used throughout Chapter 5 to drive the backend independently of the desktop client.

3.3 Technologies used

Table 3.2 lists every technology used in the project, grouped by category, as a quick reference alongside the more detailed justification given above and in Section 4.6.2.

Table 3.2: Technologies used, by category.

Category	**Technologies**
Languages	C# (.NET Framework 4.8), Python
Desktop client	WPF, MVVM, CommunityToolkit.Mvvm, XAML, Newtonsoft.Json
Backend / API	FastAPI, uvicorn
Machine learning	scikit-learn [4] (Isolation Forest [5], K-Means [6]), pandas
Data processing	`asammdf` [7] (MF4 decoding), `python-can` (hardware interface)
Persistence	SQLite
AI explanation	Google Gemini API, Ollama (local LLM fallback)
Export	PDFsharp
Documentation & diagrams	LaTeX, PlantUML
Version control	Git

3.4 Overall architecture

CANvisionNative is built as two cooperating processes on the same machine: a Windows desktop client and a local Python backend, communicating over HTTP on `localhost`. Figure 3.1 shows the layered view of the system.

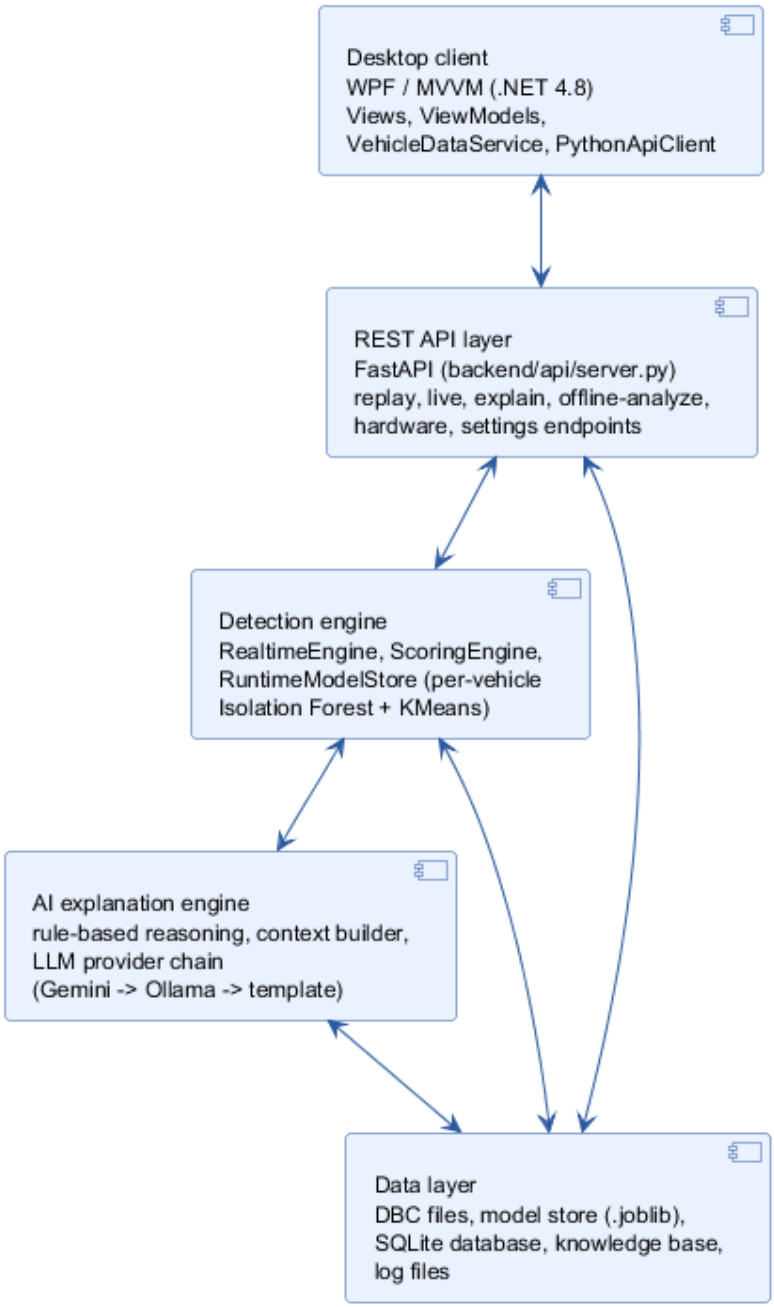

Figure 3.1: Overall layered architecture of CANvisionNative.

The desktop client only talks to the REST API layer. The API layer routes requests to the detection engine and to the data layer, and the detection engine calls the AI explanation engine when an alert explanation is requested. This separation means the detection logic can be exercised directly through the API, independently of the desktop client, which is how a large part of the validation in Chapter 5 was carried out.

3.4.1 Frontend architecture

The desktop client follows the MVVM pattern. Each screen (Home, Dashboard, Telemetry, Diagnostics, Log Playback, Anomaly Intel, Settings) has a XAML view and a corresponding

ViewModel in `ViewModels/SectionViewModels.cs`.

Two services sit between the ViewModels and the backend, as shown in Figure 3.2. `PythonApiClient` is a thin HTTP client wrapping every backend endpoint. `VehicleDataService` is the shared, in-memory client state (current snapshot, alert history, replay runtime metrics); it owns a polling timer that periodically calls the backend and raises events the ViewModels subscribe to.

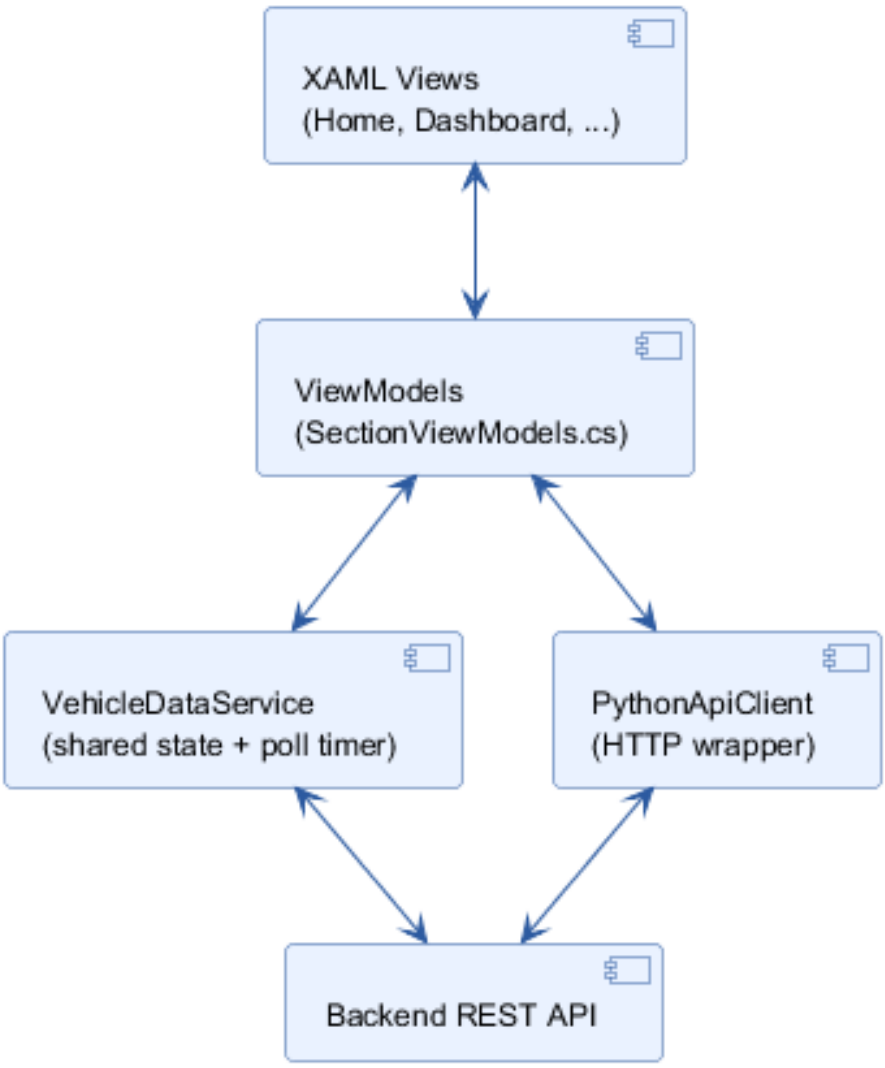

Figure 3.2: Frontend architecture: MVVM views bound to ViewModels, backed by two shared services.

Navigation between sections is driven by a section catalog and a navigation service, so that adding a new screen does not require changes to existing screens.

3.4.2 Backend architecture

The backend is a single FastAPI application (`backend/api/server.py`) organized around five responsibilities:

1. **Decoding** (`backend/decoding/`) — vehicle profile registry and DBC-based signal decoding.
2. **Data processing** (`backend/data_processing/`) — log parsing and MF4-to-CSV conversion.
3. **Machine learning** (`backend/ml/`) — feature engineering, the runtime detection engine, training scripts, and the AI explainer.
4. **Hardware** (`backend/hardware/`) — the live CAN adapter interface and session recorder.
5. **Persistence** (`backend/db/`) — the SQLite session and alert store.

Long-running work (a replay, an offline analysis) is executed in a background subprocess or thread so that the API stays responsive; the client polls a status endpoint until the work completes.

3.4.3 Database

The backend keeps a small SQLite database (`backend/db/canvision.db`) for session bookkeeping and historical alerts, independent of the per-replay summary files used for the live client views. Figure 3.3 shows its three tables.

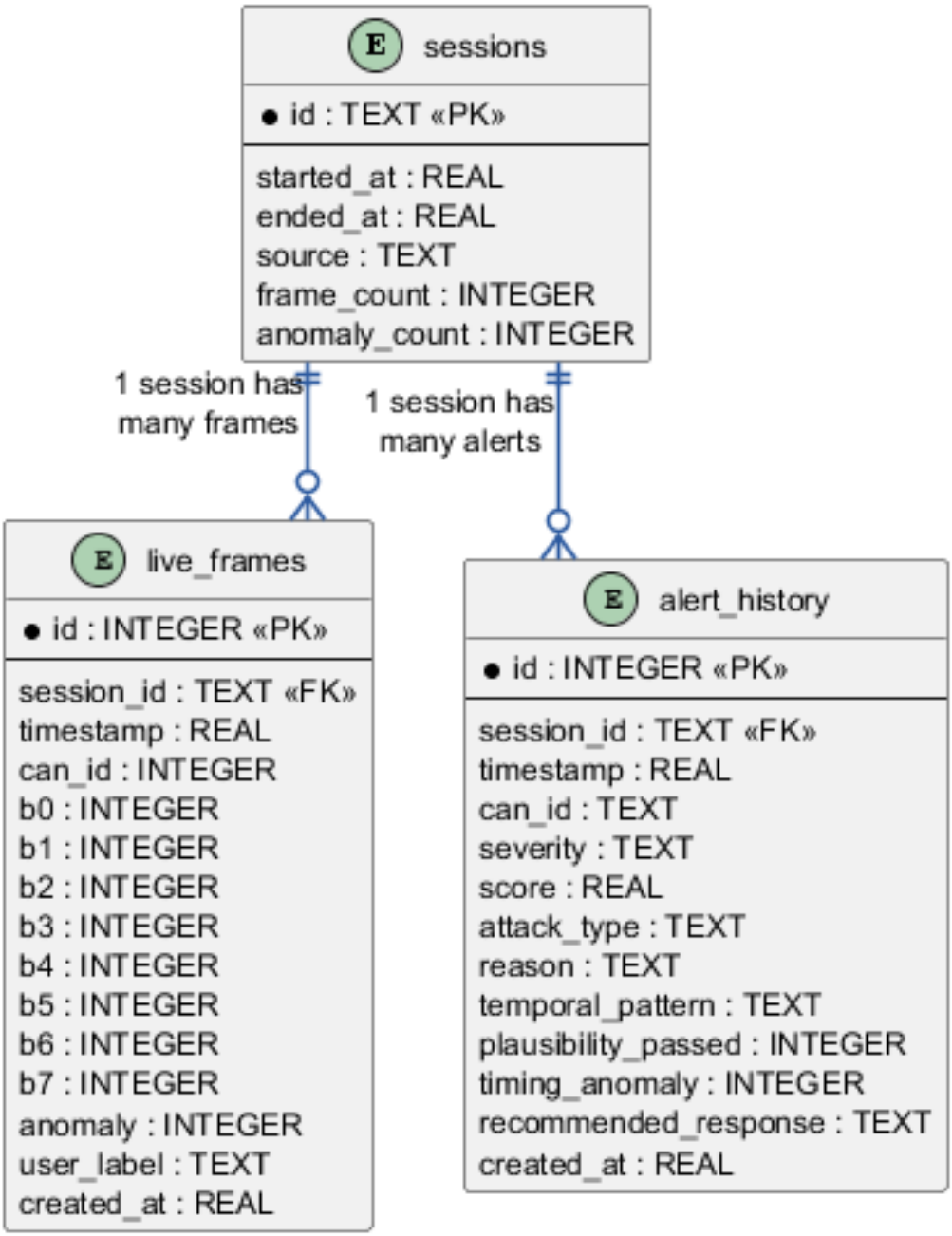

Figure 3.3: Entity-relationship diagram of the SQLite persistence layer.

A `session` groups the frames and alerts produced by one live, replay, or injection run. `live_frames` keeps every captured frame with its raw bytes and the engine's normal/anomalous classification, including an optional `user_label` so a human reviewer can correct the label for future retraining. `alert_history` keeps one row per raised alert, including the parsed detection-layer reason string and the recommended response, indexed on `session_id` and `severity` for fast filtering.

3.5 REST API

The backend exposes 49 endpoints under a small number of prefixes: `/api/replay/*` (start, stop, status, alerts), `/api/live/*` (simulation and hardware capture), `/api/explain/{index}` (AI explanation), `/analyze` (offline batch analysis), `/api/vehicles` (profile registry), `/api/session/*` (recording), `/api/db/*` (database access), and `/api/settings`. The full endpoint list is given in Appendix A. All endpoints return JSON and are consumed exclusively by the desktop client over `localhost`.

Figure 3.4 groups these endpoints by the backend component they ultimately reach: the replay, live, and offline-analysis endpoints all drive the detection engine; the explanation endpoint drives the AI explanation engine; and the vehicle, session, and settings endpoints read and write the data layer directly.

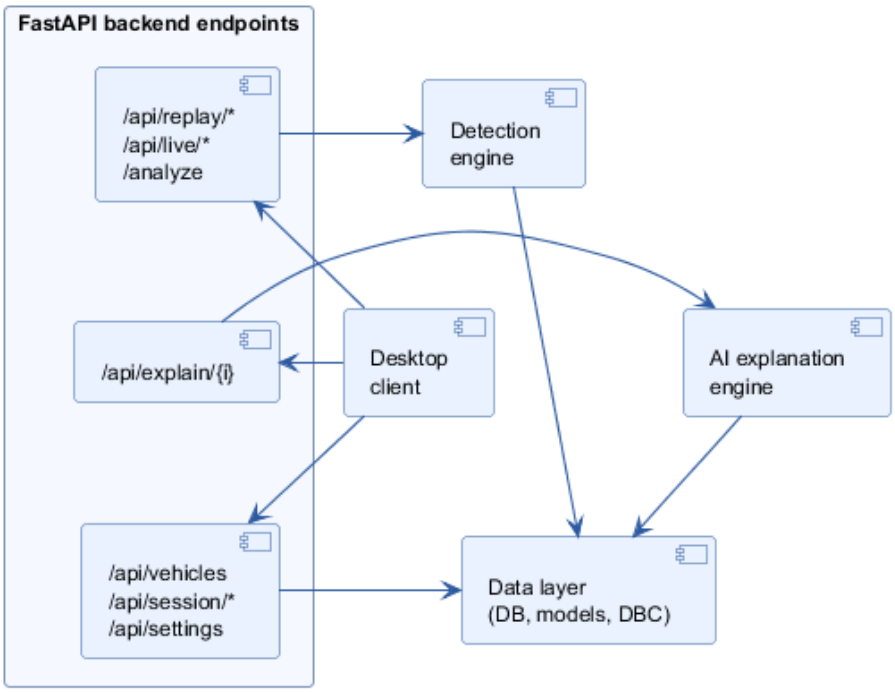

Figure 3.4: REST API flow: client requests grouped by the backend component they reach.

3.6 Replay engine

The replay engine (`backend/ml/runtime/replay_runner.py`) is launched as a subprocess by the `/api/replay/start` endpoint. It reads the input file through a single, format-aware loader (Section 4.5.2) shared with the offline batch analyzer, so that every supported format enters the pipeline as the same canonical `(timestamp, can_id, payload)` stream. Each frame is then passed, in order, to the same `RealtimeEngine` used by the live and simulated paths, guaranteeing that replay, live, and simulated detection behave identically for the same input. Figure 3.5 shows the full replay lifecycle as a sequence diagram. Section 4.5.1 describes its implementation in detail, and Chapter 5 documents two defects that were found and fixed in this exact module.

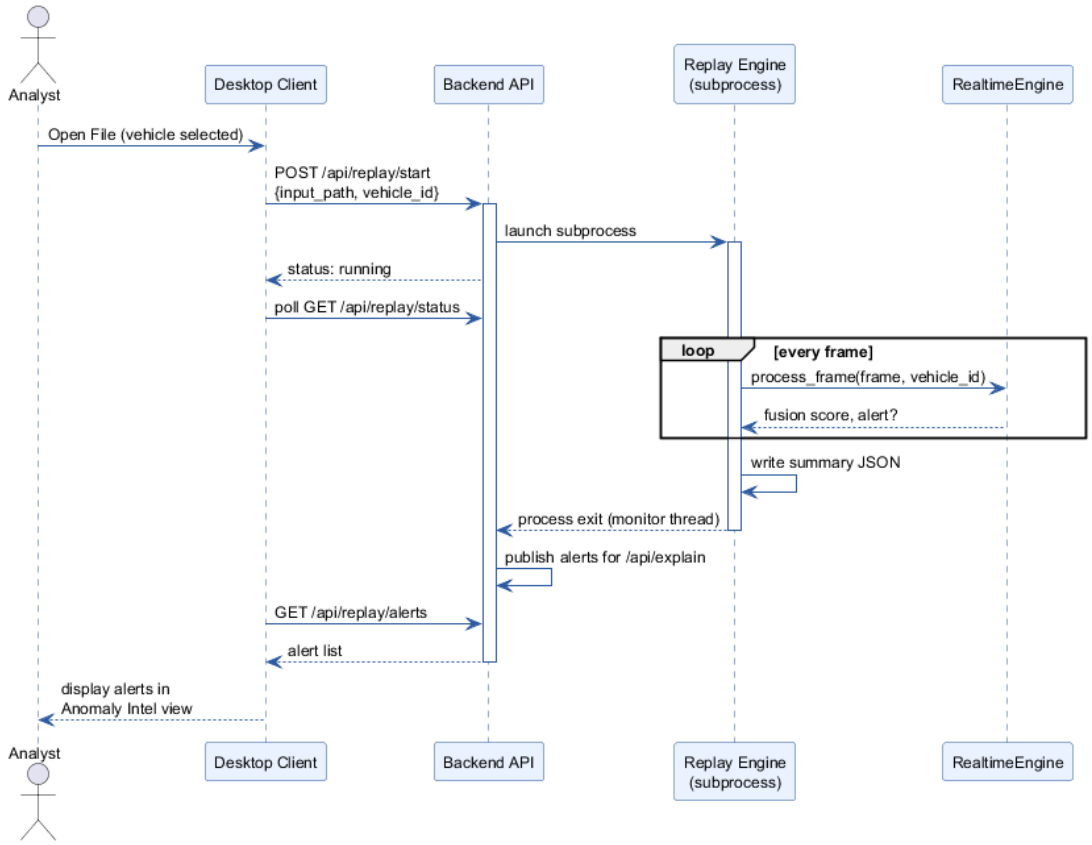

Figure 3.5: Sequence diagram: replay lifecycle, from client request to published alerts.

3.7 Detection pipeline

Figure 3.6 shows the per-frame detection pipeline, common to replay, simulation, and live hardware capture.

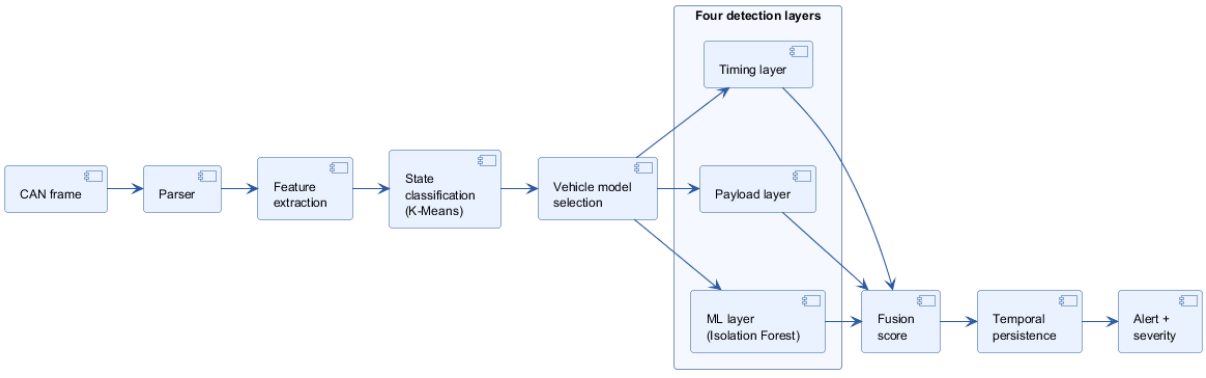

Figure 3.6: Per-frame detection pipeline: feature extraction, state classification, model selection, four-layer scoring, fusion, and alerting.

Feature extraction computes, per CAN identifier, a rolling set of statistics: inter-arrival time and its variance, message frequency, byte-level difference from the previous frame on the same identifier, and Shannon entropy of the identifier stream and of the payload. Before scoring, the

engine classifies the vehicle's current operating state (parking, idle, driving) with a K-Means model, shown as a state diagram in Figure 3.7, and selects the Isolation Forest trained for that state and for the currently selected vehicle profile.

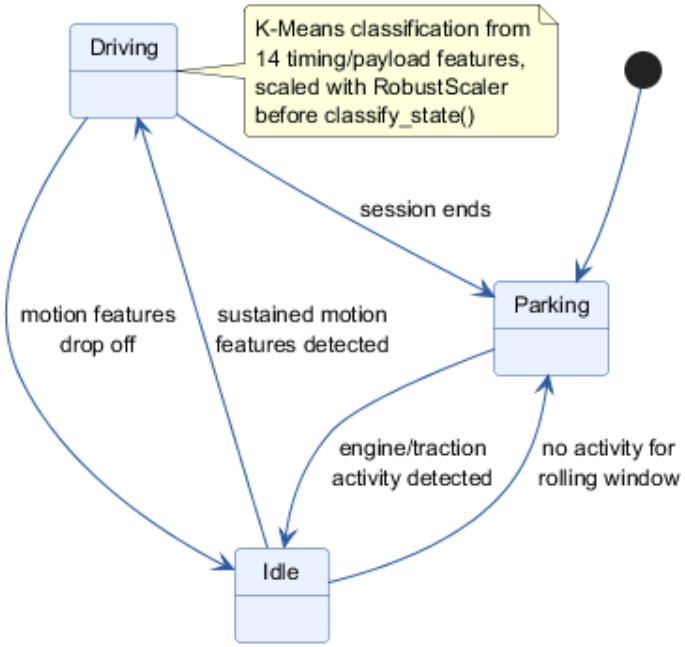

Figure 3.7: Vehicle driving-state classification: Parking, Idle, and Driving, classified by K-Means on 14 rolling features after RobustScaler normalization.

The four detection layers are then computed:

- The **timing layer** flags both abnormally bursty timing and abnormally high message frequency (a pure timing-variance metric alone would miss a steady flood, since a perfectly periodic flood has almost no timing variance).
- The **payload layer** flags abrupt, out-of-pattern byte changes.
- The **ML layer** scores the feature vector with the Isolation Forest selected for the classified state and vehicle profile, falling back to a lightweight heuristic if no matching per-vehicle model is available.
- The **temporal-persistence layer** tracks, over a rolling window, how often the fused score has stayed above the high-severity threshold, so an isolated single-frame spike is treated differently from a sustained anomaly.

The four layers are combined into one fusion score by a fixed weighted sum, and the fused score is compared against four severity thresholds (LOW, MEDIUM, HIGH, CRITICAL). Only frames whose fused score crosses the configured alert threshold are turned into alerts, subject to a per-identifier debounce so that a single burst is not reported as dozens of near-duplicate alerts. The exact weight and threshold values are listed in Table B.3 (Appendix B).

3.8 Hardware layer

The hardware layer (`backend/hardware/can_interface.py`) wraps the `python-can` library to support three transports: ELM327 over a serial/slcan connection, PEAK PCAN-USB, and native SocketCAN. It exposes connect, disconnect, and status endpoints, and feeds every received frame through the same `RealtimeEngine` instance used by replay, with the vehicle identifier selected in the Settings view. This design means the detection pipeline itself required no change to support live hardware once an adapter is available; only the transport layer differs, which is why the code-complete-but-hardware-blocked status described in Chapter 2 was an acceptable risk to carry through the project.

3.9 AI explanation engine

Figure 3.8 shows the explanation workflow triggered when the analyst selects an alert.

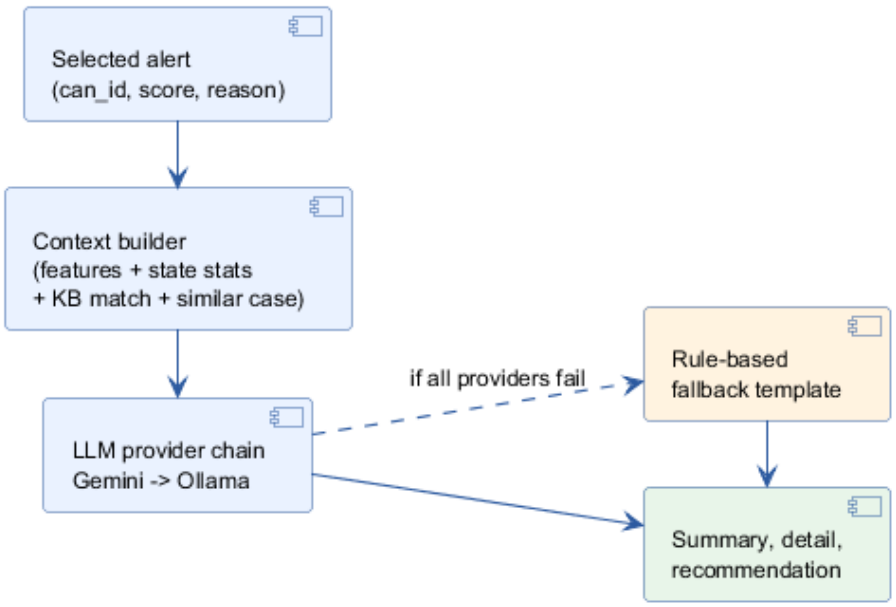

Figure 3.8: AI explanation workflow: context building, LLM provider chain, and rule-based fallback.

The context builder assembles four sources before calling the language model: the current alert's own features (identifier, frequency, entropy, fusion-layer scores), the expected statistics for the vehicle's current state, a keyword match against a small attack knowledge base, and the most similar historical anomaly found by cosine similarity against a persistent, growing store of previously explained anomalies. The assembled context is sent to a language-model provider chain — Gemini first, a local Ollama model second — and, if neither is reachable, a rule-based template still produces a complete, correctly parameterized explanation instead of failing the request, satisfying requirement NFR6.

3.10 Class diagram (backend detection core)

Figure 3.9 shows the simplified class diagram of the backend's detection core, the part of the system most directly involved in the validation work of Chapter 5.

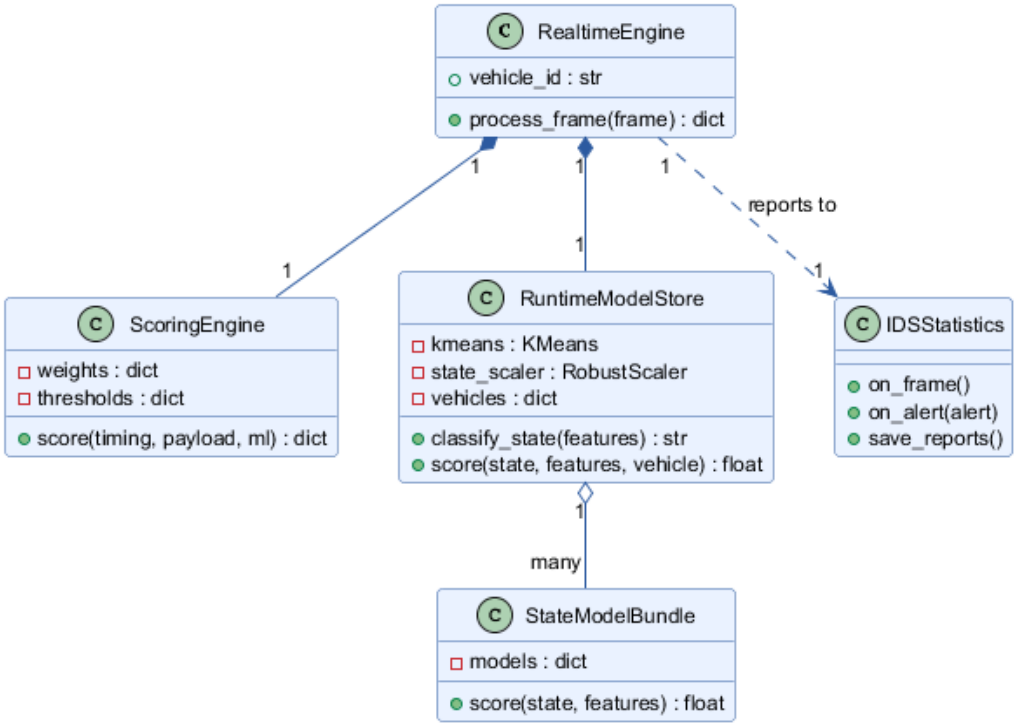

Figure 3.9: Simplified class diagram of the backend detection core.

`RealtimeEngine` is the single entry point used by replay, live simulation, and hardware capture. It owns the current `vehicle_id`, delegates timing/payload scoring and fusion to `ScoringEngine`, and delegates ML scoring to `RuntimeModelStore`, which holds one `StateModelBundle` per vehicle profile (plus a generic fallback bundle) and is responsible for selecting the correct per-state, per-vehicle model. `IDSStatistics` accumulates per-run counters and severity/alert-reason distributions and writes the final summary report consumed by both the desktop client and the AI explanation engine.

3.11 Deployment diagram

Figure 3.10 shows how the system is deployed on a single workstation, with the two external systems it may reach over the network.

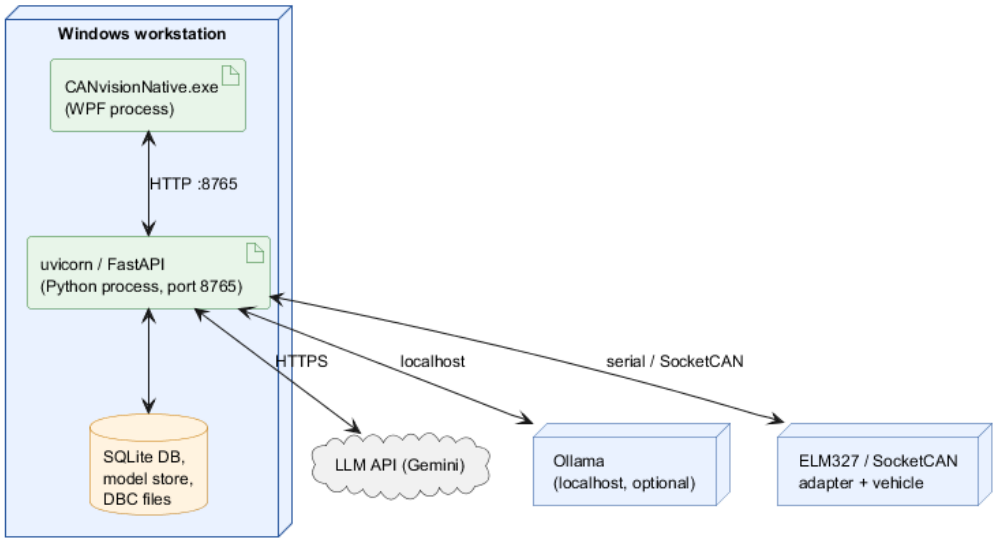

Figure 3.10: Deployment diagram: single-workstation deployment with external LLM and hardware links.

3.12 Conclusion

This chapter presented the layered architecture of CANvisionNative, its desktop and backend components, its database schema, its REST API, the replay engine, the four-layer detection pipeline, the hardware transport layer, and the AI explanation workflow, together with the class, entity-relationship, and deployment diagrams that summarize the design. The next chapter describes how this design was implemented.

Chapter 4

Implementation

4.1 Introduction

This chapter describes how the design presented in Chapter 3 was implemented. It covers the development environment, the desktop application, the backend, the replay engine and parsers, the MF4 conversion path, the vehicle profile system, the machine-learning pipeline, the AI explanation module, the charting and export features, the settings module, and the hardware layer. Only short, illustrative snippets are shown; full listings are not included, in line with the reporting guidelines followed for this document.

4.2 Development environment

- **Hardware.** A standard Windows desktop/laptop workstation was used for development, training, and validation; no specialized hardware was required for the software work. The live hardware capture path targets an ELM327 (serial/slcan), a PEAK PCAN-USB adapter, or native SocketCAN, but validating this path end to end requires a physical adapter connected to a vehicle, which did not arrive during the project (Chapter 5, Limitations).
- **Operating system.** Windows 10/11, the target platform for the WPF desktop client.
- **Languages.** C# (.NET Framework 4.8) for the desktop client; Python for the backend.
- **Key libraries.** WPF and CommunityToolkit.Mvvm for the client; FastAPI and uvicorn for the REST API; scikit-learn [4] for Isolation Forest and K-Means; pandas and `asammdf` for data processing and MF4 decoding [7]; `python-can` for the hardware layer; PDFsharp (via `PdfReportBuilder.cs`) for PDF export.
- **Tooling.** Visual Studio for the C# client; a standard Python virtual environment for the backend; SQLite as the embedded database, requiring no separate server process.

4.3 Repository organization

The two halves of the system live in the same repository but keep separate top-level folders, reflecting the process boundary between them (Figure 4.1).

The C# client's code is split into `UI/` (XAML views), `ViewModels/` (the single `SectionViewModels.cs` file), `Services/` (`PythonApiClient`, `VehicleDataService`, `PdfReportBuilder`), `Models/`, and `Converters/`.

The Python backend is split by responsibility under `backend/`: `api/` (the FastAPI entry point), `decoding/` (vehicle profiles, DBC decoding), `data_processing/` (parsers, MF4 conversion), `ml/` (feature engineering, the runtime engine, training scripts, the AI explainer), `hardware/` (the live adapter interface), `db/` (the SQLite store), and `knowledge/` (the attack knowledge base and signal map).

Trained models, DBC files, and datasets live under `assets/`, outside both source trees.

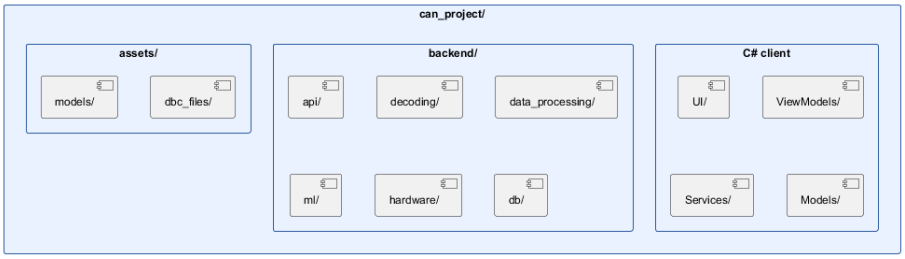

Figure 4.1: Repository organization: the WPF client, the Python backend, and shared assets.

Table 4.1 gives the size of each side of the repository, measured directly on the source tree (excluding generated build output, downloaded datasets, and third-party packages).

Table 4.1: Repository statistics (source code only).

Component	Files	Lines
Python backend (`backend/`)	135	18,363
C# client (`UI/`, `ViewModels/`, `Services/`, `Models/`, `Converters/`)	33	10,084
XAML views (`UI/*.xaml`)	13	—

4.4 WPF application

The desktop client is a WPF application targeting .NET Framework 4.8, structured with the MVVM pattern and CommunityToolkit.Mvvm for command and observable-property boilerplate. The main window hosts a navigation bar (Home, Dashboard, Telemetry, Diagnostics, Log Playback, Settings, Anomaly Intel) backed by a section catalog, so that each section is a self-contained view/ViewModel pair. All ViewModels live in a single file, `ViewModels/SectionViewModels.cs`, an explicit project convention kept throughout development to avoid navigation wiring being duplicated across files.

Two services are shared by every ViewModel:

- `PythonApiClient` wraps every backend call behind a typed method (for example `StartReplayAsync`, `GetAlertExplanationAsync`), using `HttpClient` and Newtonsoft.Json for (de)serialization.
- `VehicleDataService` holds the client-side state shared across views: the current snapshot, the alert history, and the replay/live runtime metrics. It runs a polling loop (`OnRefreshTick`) that periodically asks the backend for replay status, partial alerts, and live signals, and raises events (`AlertHistoryUpdated`, `DataUpdated`) that ViewModels subscribe to.

Figure 4.2 shows the Home screen, the client's entry point: it reports whether the backend is reachable and lets the analyst choose between a live session and an offline import.

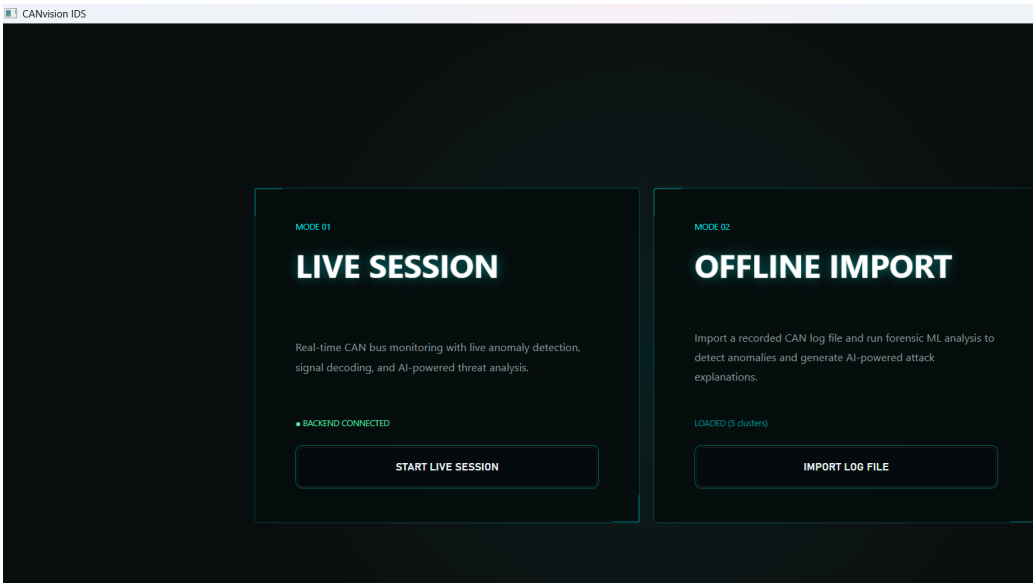

Figure 4.2: Home screen — Live Session / Offline Import choice, backend connection status.

Figure 4.3 shows the Log Playback view during a completed replay of the real Kia EV6 recording used throughout Chapter 5: transport controls, vehicle profile selector, detection-analysis chart, and replay progress panel.

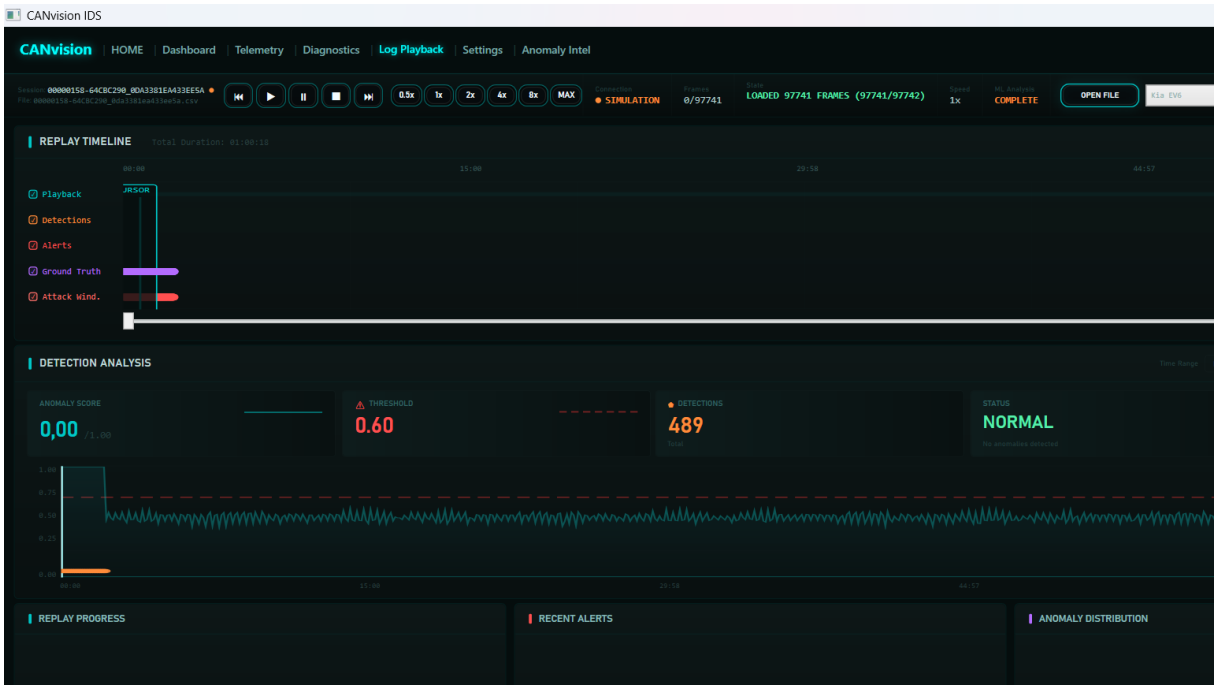

Figure 4.3: Log Playback view after a completed replay of the 97,741-frame Kia EV6 recording.

Figure 4.4 shows the Anomaly Intel view: the alert list on the left, the four-layer attribution bars for the selected alert, and the Chat with AI panel on the right, showing a freshly generated explanation for the selected alert.

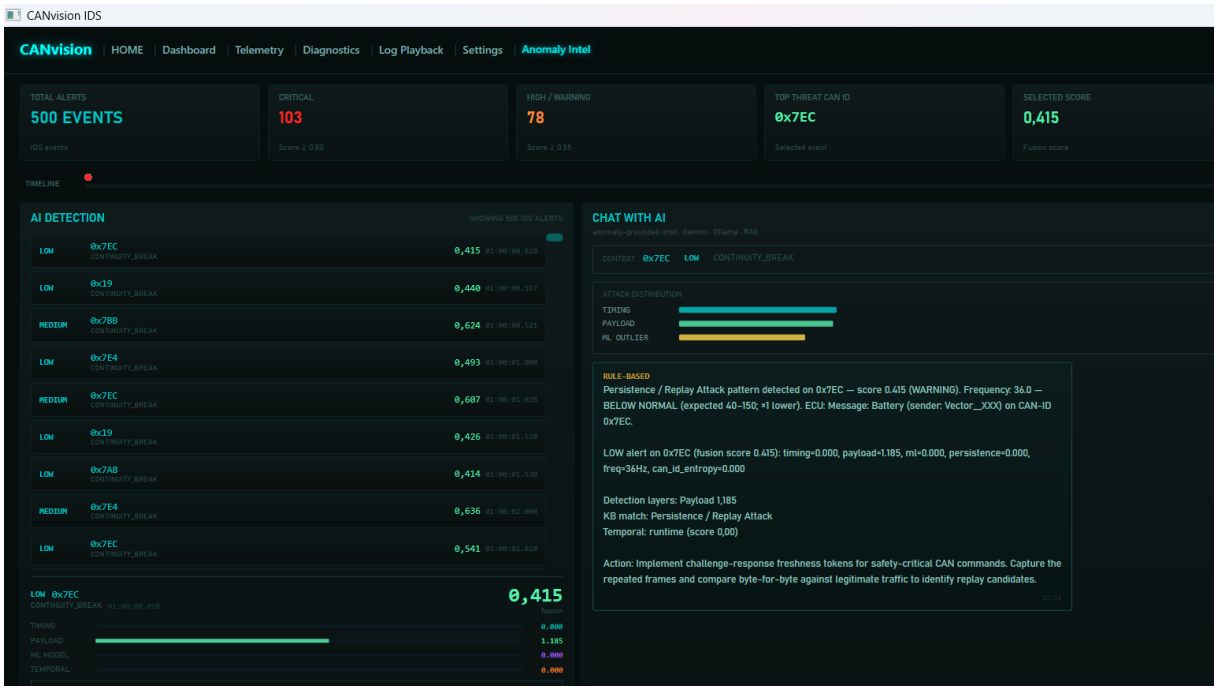

Figure 4.4: Anomaly Intel view: alert list, layer-attribution bars, and the Chat with AI explanation panel.

The remaining two live views, Telemetry and Diagnostics, are shown in Figure 4.5 and Figure 4.6. Telemetry draws the decoded signal channels (speed, state of charge, motor temperature,

voltage) alongside the raw live signal monitor and per-identifier activity ranking; Diagnostics exposes adapter and bus health, a running IDS event log, and a set of recovery actions for the live session.

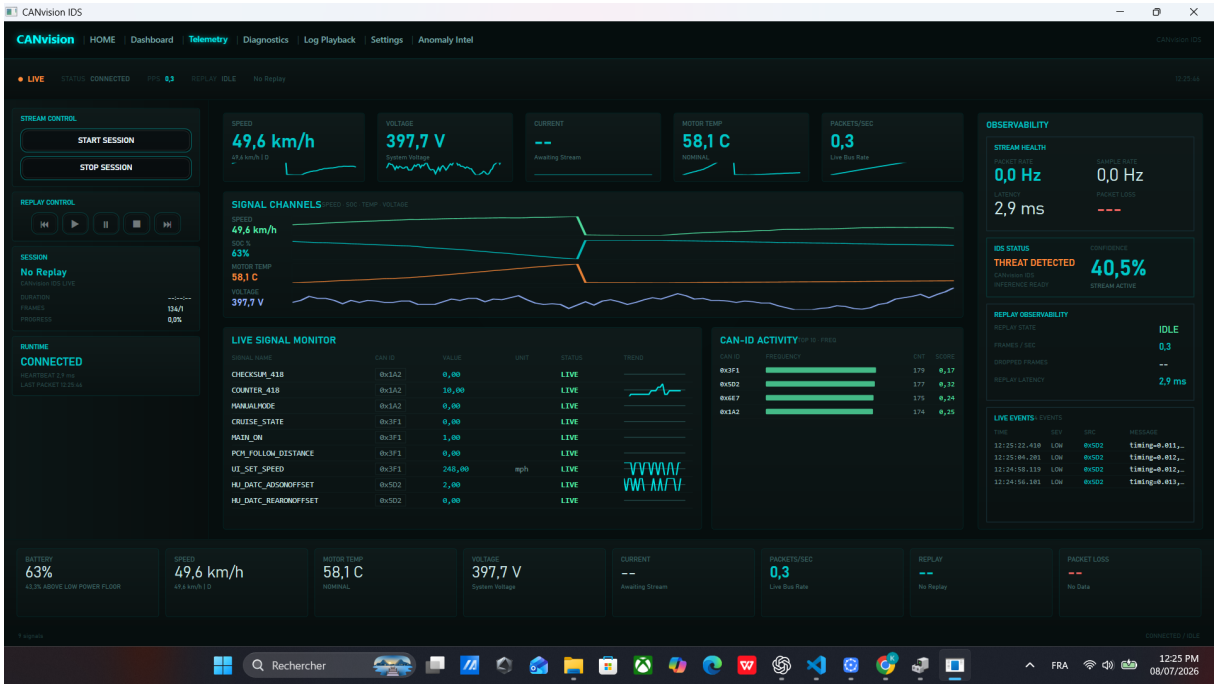

Figure 4.5: Telemetry view: decoded signal channels, live signal monitor, and CAN-ID activity ranking.

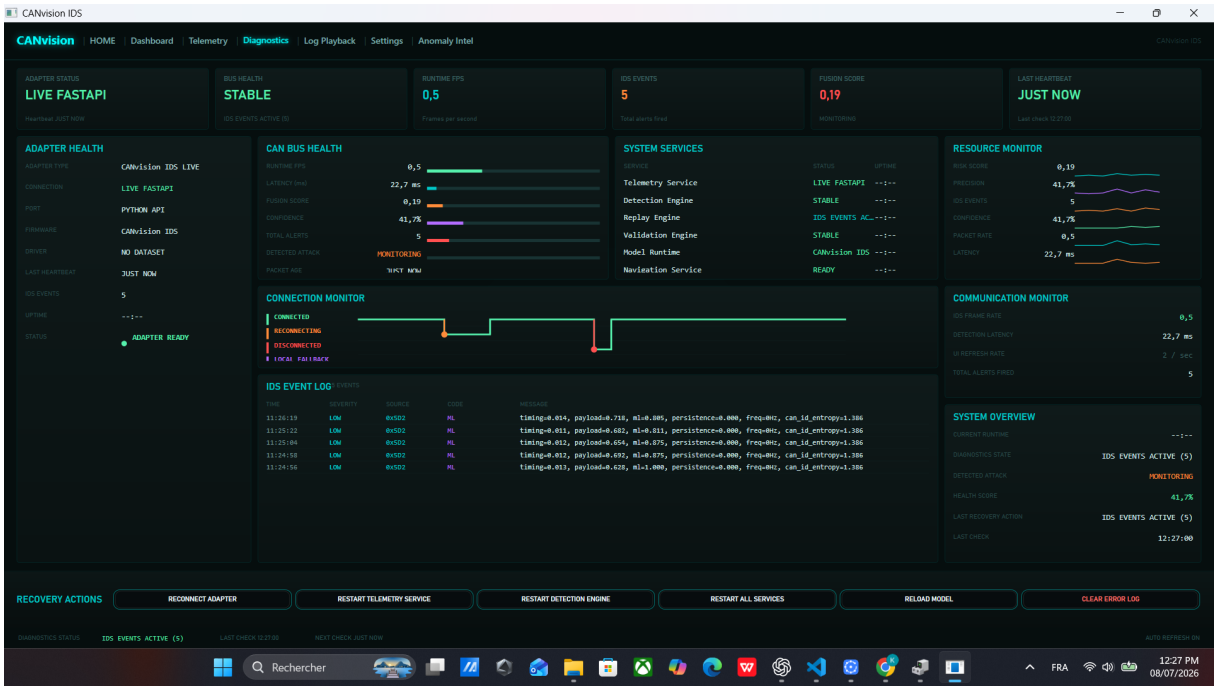

Figure 4.6: Diagnostics view: adapter/bus health, IDS event log, and recovery actions.

4.5 FastAPI backend

The backend entry point, `backend/api/server.py`, defines all REST endpoints in a single FastAPI [8] application. Long-running operations, such as starting a replay or running an offline batch analysis, are launched as a subprocess or background thread, and the endpoint returns immediately with a status the client can poll. Module-level, process-lifetime state (the current replay process handle, the most recent alert context list, the explanation cache) is protected by explicit locks where it is shared between the request-handling thread and a background monitoring thread.

4.5.1 Replay engine

This section describes the concrete implementation of the replay engine introduced in Section 3.6. `backend/ml/runtime/replay_runner.py` is invoked as a subprocess by `POST /api/replay/start`. It loads the input file through the shared, format-aware loader described in Section 4.5.2, builds three aligned arrays (timestamps, identifiers, payloads), and feeds them one by one into a `RealtimeEngine` instance configured with the requested vehicle identifier. Every 1,000 frames it writes a partial-alerts file to disk so the client can show live progress before the run completes; at the end it writes a summary JSON file containing the full alert list and per-run statistics, which is what `GET /api/replay/alerts` serves to the client.

A background thread on the server side (`monitor_replay_process`) waits for the subprocess to exit and updates the replay status to `completed` or `error`. It also publishes this replay's own alerts into the state consumed by the AI explanation endpoint, a step added during the validation phase described in Chapter 5.

4.5.2 Parser and canonical frame representation

Every supported log format is converted, at import time, into the same canonical stream: a sequence of `(timestamp, can_id, payload)` tuples. A single dispatch function, `_load_file`, selects the correct parser by file extension:

- `.csv` — generic `timestamp, can_id, b0..b7` columns, or SavvyCAN-style columns.
- `.mf4` — ASAM MF4 [7] binary logs, decoded with `asammdf` (Section 4.5.3).
- `.asc` — Vector CAN text format.
- `.log` — candump / socketCAN text format, including the timestamped `(seconds) interface id#data` variant used by several public datasets.
- `.txt` — PCAN text format, falling back to candump parsing if the PCAN pattern is not matched.

This single dispatch point is what both the replay engine and the offline batch analyzer call, so that a defect in one format's parser cannot silently produce a different frame stream for replay than for offline analysis. Chapter 5 documents a defect found in exactly this area: before the fix, the replay engine had its own, separate, CSV-only loading code that ignored this shared dispatcher.

4.5.3 MF4 conversion

MF4 files recorded by multi-channel CANedge-style loggers can spread real CAN traffic across several channel groups, most of which are empty template groups. The conversion code (`backend/data_processing/mf4_to_csv_converter.py`) iterates every group in the file explicitly and merges the populated ones, instead of relying on the library's default single-dataframe view, which only surfaces the first group and silently drops the rest. This function is shared between the standalone MF4-to-CSV conversion tool and the format-aware parser described above, so both code paths see the same, complete set of frames.

4.5.4 Vehicle profile system

`backend/decoding/vehicle_profiles.py` defines a small registry of vehicle profiles (Table 4.2), each mapping a vehicle identifier to a bitrate, a UDS-response DBC file (used to decode diagnostic responses), and, where available, a raw broadcast DBC file (used to decode spontaneous, non-diagnostic frames).

Table 4.2: Supported vehicle profiles.

Identifier	Display name	Bitrate
`nissan_leaf`	Nissan Leaf (2019, 40/62 kWh)	500 kbps
`hyundai_ioniq5`	Hyundai Ioniq 5	500 kbps
`hyundai_kia`	Hyundai / Kia (generic)	500 kbps
`skoda_enyaq`	Škoda Enyaq iV (VW MEB)	500 kbps
`vw_skoda_audi`	VW / Škoda / Audi (generic)	500 kbps
`renault_zoe`	Renault Zoe	500 kbps
`tesla_model3`	Tesla Model 3	500 kbps
`kia_ev6`	Kia EV6	500 kbps
`bmw_i3`	BMW i3 (2017, 60 Ah REX)	500 kbps

Selecting a profile in the client sets the `vehicle_id` sent to every replay and live-capture request, which in turn selects both the DBC decoding tables and the trained model bundle used for scoring, as described next. The nine profiles in Table 4.2 are the ones with full signal decoding through a named DBC file; the trained model store (Appendix C) additionally holds model bundles for training-only sources without commercial vehicle names, such as the ROAD and Car-Hacking benchmark datasets, which is why the model store has more entries than Table 4.2.

4.6 Machine learning pipeline

This section covers the full machine-learning side of CANvisionNative: how a raw frame becomes a feature vector, how the learning algorithms were chosen, how the models are trained, and how they are used at runtime.

4.6.1 Feature engineering

For every CAN identifier, the engine maintains a small rolling window of recent inter-arrival times and byte-level differences. From this window it derives fourteen features at each frame: the current and rolling-mean/variance inter-arrival time, the current and rolling byte difference, message frequency and rate, a burstiness ratio, the coefficient of variation of inter-arrival time, changed-byte count, and the Shannon entropy of both the identifier stream and the payload bytes. Every feature is clipped to a fixed range and passed through `log1p` before being handed to the models, matching the transform used when the models were trained. Figure 4.7 shows this as a pipeline: cleaning, the per-identifier rolling window, the three feature groups (timing, payload, entropy), and the final fourteen-value vector.

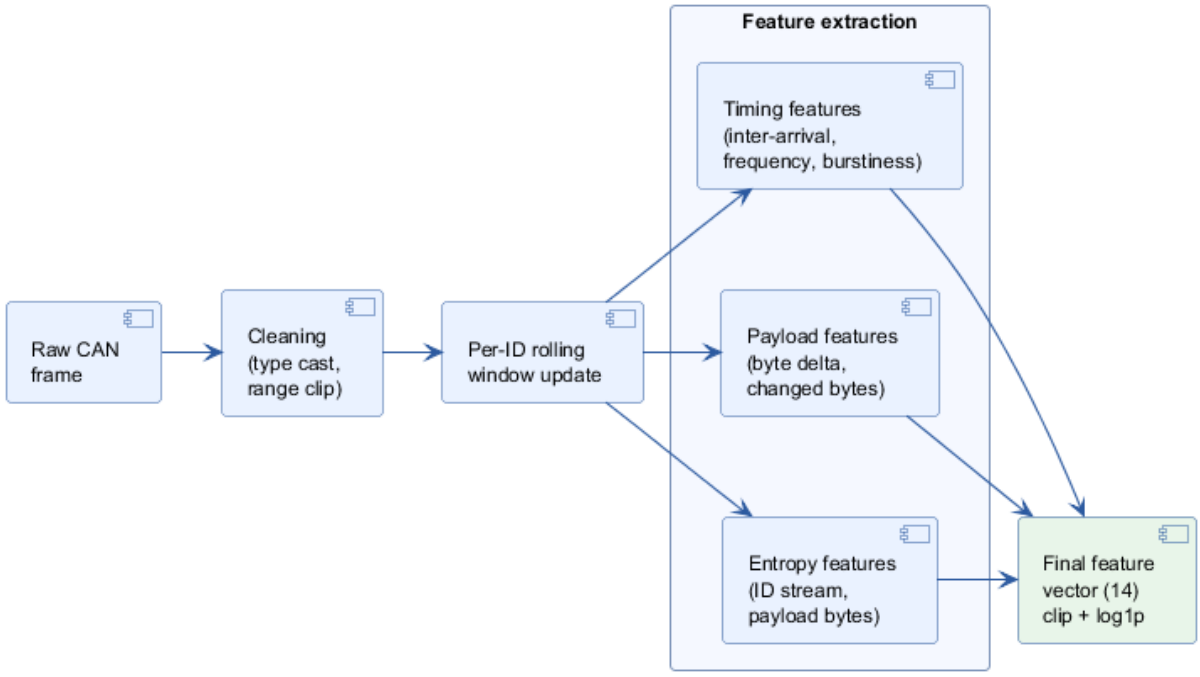

Figure 4.7: Feature engineering pipeline: from a raw frame to the final 14-feature vector.

4.6.2 Machine learning model selection

4.6.2.1 Problem definition

The core learning problem is to decide, for a given vehicle and a given driving state, whether a frame's features look normal or anomalous, without access to a reliable set of labelled attacks for every vehicle. A second, smaller learning problem is to decide which driving state (parking, idle, driving) the vehicle is currently in, so that the right anomaly model is used.

4.6.2.2 Supervised versus unsupervised learning

A supervised approach would need a labelled dataset of normal and attack frames for every vehicle profile CANvisionNative supports. This was rejected as the main detection strategy for three reasons. Most of the training corpus (Appendix C) is normal, unlabelled traffic gathered from real vehicles and community logs; only a small part (ROAD, Car-Hacking) carries attack labels, and those labels come from a different vehicle and a different attack methodology than any vehicle an analyst might import. A supervised model trained on one manufacturer's attack patterns is also unlikely to generalize to an attack style it has never seen, which defeats the purpose of an anomaly-detection layer meant to catch unknown attacks. And, most practically, gathering real attack traffic on every supported vehicle was not feasible within this project. An unsupervised, anomaly-based approach avoids all three problems: it only needs a sample of normal traffic for the vehicle it is trained on, which the training corpus provides in quantity.

4.6.2.3 Candidate algorithms

Table 4.3 compares the unsupervised and supervised candidates considered for the anomaly-scoring layer.

Table 4.3: Candidate machine-learning algorithms for anomaly scoring.

Algorithm	**Advantages**	**Disadvantages**
Isolation Forest	Fast to train and to score; no distance computation, so it scales well with more features; does not assume a particular data distribution; works well on the kind of tabular, numeric features produced by CAN feature engineering.	Less interpretable than a hand-written rule; needs a contamination parameter to be set.
One-Class SVM	Well-studied, clear geometric interpretation (a boundary around normal data).	Training cost grows quickly with dataset size; sensitive to kernel and parameter choice; slower to score in a real-time loop.
Local Outlier Factor (LOF)	Captures local density variations, useful when normal traffic has several distinct clusters.	Expensive at inference time (distance to neighbours must be recomputed), which conflicts with the per-frame, real-time scoring requirement (NFR1).
DBSCAN	No need to pre-specify the number of clusters; can find arbitrarily shaped clusters.	Designed for offline clustering, not per-sample scoring; does not naturally give a continuous anomaly score for a new point.
Random Forest (supervised)	Strong accuracy when labelled data is available; interpretable feature importance.	Requires labelled attack data per vehicle, which is not available at the scale needed (see above).
Autoencoder	Can model complex, non-linear normal patterns given enough data.	Needs more training data and tuning effort per vehicle than the corpus reasonably supports; harder to justify and explain to a non-specialist analyst; heavier at inference time than a tree-based model.
K-Means	Simple, fast, easy to explain (cluster centroids); works well for a small, well-separated number of operating states.	Assumes roughly convex clusters;

4.6.2.4 Selection criteria and engineering justification

Three criteria drove the final choice. The algorithm had to score a single frame fast enough to sustain real-time replay and live capture (NFR1). It had to work with only normal training data, per the supervised-versus-unsupervised discussion above. And its behaviour had to be explainable to an analyst without a machine-learning background (NFR3), at least at the level of "how anomalous is this compared to what this vehicle normally does".

Isolation Forest was selected for the anomaly-scoring layer because it satisfies all three: it scores a single feature vector in microseconds, it trains directly from normal traffic without needing labelled attacks, and its anomaly score has a simple interpretation (how easy the point was to isolate from the rest of the data). One-Class SVM and LOF were both considered and rejected mainly on the first criterion: LOF's per-sample cost scales with the size of the reference dataset, which conflicts with sustained real-time scoring, and One-Class SVM's training cost becomes a problem when retraining on a multi-million-frame corpus. DBSCAN was rejected because it does not naturally produce a per-sample anomaly score, only cluster membership. The Autoencoder was rejected mainly on the third criterion combined with training cost: justifying and debugging a neural model's output to an analyst, and to this report's jury, is harder than justifying an Isolation Forest's path-length score, for a gain in accuracy that was not demonstrated to be necessary for this problem.

K-Means was selected for the separate driving-state classification step, not as an anomaly detector, because the underlying problem is a genuine clustering problem with a small, known number of expected states (parking, idle, driving) that separate reasonably well on timing and payload features. Its simplicity also matters operationally: it is the same algorithm whose state-classification defect is documented in Chapter 5, and a simple, inspectable model was part of what made that defect possible to diagnose precisely instead of remaining a mysterious accuracy problem.

4.6.2.5 The contamination parameter

Isolation Forest requires a `contamination` parameter, an estimate of the expected proportion of anomalies in the training data, used to convert raw path-length scores into a binary normal/anomalous decision at training time. Because the training corpus (Appendix C) is overwhelmingly normal traffic, contamination was set low and treated as a training-time calibration constant rather than a detection threshold: the detection threshold an analyst actually configures (Table B.3, Appendix B) is applied later, to the fused, remapped 0–2 score described in Section 3.7, not to the raw Isolation Forest output directly.

4.6.3 Machine learning training pipeline

Figure 4.8 shows the offline training pipeline that produces every vehicle's model bundle: cleaning and normalizing the raw corpus, running it through the same feature engineering used at runtime, splitting into training and validation subsets, training the shared K-Means state classifier, training one Isolation Forest per state and per vehicle, validating against the held-out split, and

exporting the fitted models to `.joblib` files that the runtime store loads on startup.

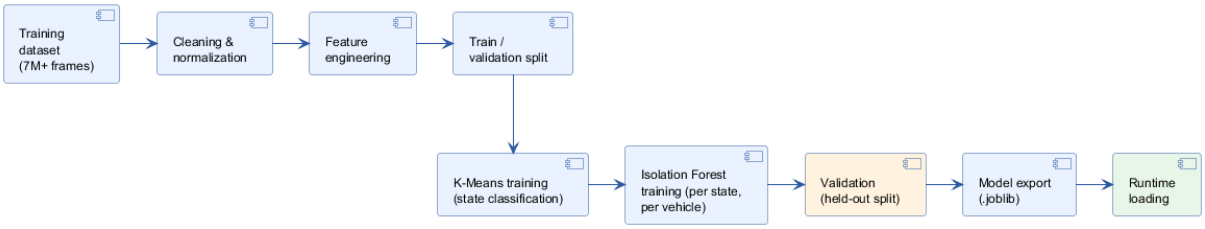

Figure 4.8: Machine learning training pipeline, from the raw corpus to the runtime model store.

4.6.3.1 Validation methodology

Each trained Isolation Forest was checked against a held-out split of its own training corpus before export, confirming that the fitted contamination level produced a normal/anomalous split consistent with the training-time expectation, and that scoring a held-out batch did not raise a shape or scaling error. This is a model-level sanity check, distinct from the system-level validation described in Chapter 5: it confirms the model file itself is usable, not that the runtime engine calls it correctly. Chapter 5's Fix 1 and Fix 2 are precisely examples where a model passed this model-level validation perfectly, while the runtime code around it still called it incorrectly — which is why both levels of validation were necessary.

4.6.4 Isolation Forest scoring

Each vehicle profile that has a trained model bundle stores one scikit-learn [4] Isolation Forest [5] per driving state (for example, *idle* and *driving*), each with its own fitted `RobustScaler`. Scoring a frame means: scale the feature vector with that state's scaler, call `decision_function` on the forest, and linearly remap the result so that the model's own decision threshold maps to a score of 0.5, and increasingly negative (more anomalous) values map towards 1.0. If no bundle exists for the requested vehicle, the store falls back to a generic bundle, and if even that has no model for the current state, the engine falls back to a lightweight heuristic derived from the timing and payload layers, so that scoring never fails outright.

4.6.5 State classification

Before scoring, the engine must decide which driving state the vehicle is currently in, since a different Isolation Forest is trained per state. This is done with a K-Means [6] model shared across vehicles: the current feature vector is scaled with a dedicated `RobustScaler` and passed to `kmeans.predict`, and the resulting cluster index is mapped to a state name (*parking*, *idle*, *driving*) through a small lookup table produced at training time.

Two implementation details matter here and are directly connected to a defect documented in Chapter 5: the feature vector passed to the state classifier must have the exact same length and order as the one used to fit the K-Means model (fourteen features), and it must be scaled with `RobustScaler.transform` before calling `predict`, exactly as it was scaled before `fit`

during training. Skipping either step does not raise a visible error if the surrounding code catches exceptions broadly, which is exactly what made this defect hard to notice without deliberate, evidence-based testing.

4.6.6 Runtime inference

Figure 4.9 summarizes how the pieces above fit together for a single incoming frame at runtime: parsing, feature extraction, state classification, vehicle-specific model bundle selection, Isolation Forest scoring, fusion with the timing and payload layers, severity classification, and alerting.

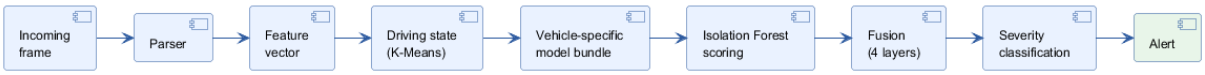

Figure 4.9: Runtime inference pipeline, from an incoming frame to an alert.

4.7 AI explanation module

The explanation module (`backend/ml/explainer/`) implements the workflow introduced in Section 3.9, and is called from the `GET /api/explain/{alert_index}` endpoint. It builds a context object from four sources: the alert's own features, expected statistics for the current vehicle state, a keyword match against a small JSON attack knowledge base, and the most similar past anomaly retrieved by cosine similarity from a persistent JSON-lines history file.

This context is then sent to a provider chain that tries Google Gemini first, then a locally running Ollama model, and finally a rule-based template. The template fills in the same fields (summary, detail, recommendation) directly from the numeric context, so an explanation is always produced even with no network access. Each resolved explanation is also appended to the historical anomaly store, so that later, similar anomalies can reference it.

Explanations are cached by a key derived from the alert's own identifier and score, to avoid asking the language model the same question twice for the same alert within a run; Chapter 5 documents how the lifecycle of this cache, not the explanation logic itself, was the source of a validation defect, and how it was corrected.

4.8 Charts

The client draws its charts (anomaly score history, per-identifier activity, severity distribution, live signal channels) with plain WPF `Polyline` elements bound to rolling `PointCollection` buffers updated on each data tick, rather than a third-party charting library, keeping the rendering path simple and dependency-free.

4.9 Export

Two export paths are implemented from the Anomaly Intel view, shown in Figure 4.10: CSV export writes the currently displayed alert list, including the parsed reason, KB match label, and explanation text where available, to a user-chosen file; PDF export (`Services/PdfReportBuilder.cs`, built on PDFsharp) renders a paginated A4 report with a header, summary statistics, and a formatted alert table, suitable for sharing outside the application. Both paths read from the same in-memory alert list the analyst is looking at, so an export always matches what is currently on screen.

Figure 4.10: Export workflow: both CSV and PDF export read the same displayed alert list.

Appendix D documents one minor, genuine formatting defect found in the CSV path during validation (an unquoted, French-locale decimal comma in the `Score` column) that is left as a known issue rather than corrected here, since the engineering scope of this project was frozen once the six defects of Chapter 5 were fixed and re-validated.

4.10 Settings

The Settings view lets the analyst configure the alert threshold, the LLM provider key (Gemini), and the hardware interface (adapter type, COM port, bit rate). Configuration is persisted to `%AppData%\CANvision\config.json` and reloaded, and any configured LLM key is applied as an environment variable at startup so the backend picks it up without a separate configuration step.

4.11 Hardware

`backend/hardware/can_interface.py` wraps `python-can` to support ELM327 (over an slcan serial connection), PEAK PCAN-USB, and native SocketCAN. Connecting sets the engine's active vehicle identifier, so live frames are scored with the correct per-vehicle model, exactly as a replay does. Disconnecting resets it to the generic profile, so a later, unrelated request (for example the synthetic `/vehicle` endpoint used for demo purposes) does not keep scoring against a vehicle that is no longer connected.

4.12 Conclusion

This chapter described the implementation of CANvisionNative's main modules: the development environment, the WPF client and its two shared services, the FastAPI backend and its replay engine, the shared canonical parser and MF4 conversion path, the vehicle profile system, the machine-learning pipeline (feature engineering, Isolation Forest scoring, and state classification), the AI explanation module, and the charting, export, settings, and hardware layers. The next chapter presents how this implementation was validated against a real vehicle recording, the defects found, and the corrections applied.

Chapter 5

Validation and Results

5.1 Introduction

Building a detection platform is only half of the engineering work. The other half is proving that it behaves correctly on real data. This chapter validates CANvisionNative itself as a piece of software, a distinct exercise from the BMS validation work the platform is built to support, as described in Chapter 1. It is the most important chapter of this report, and it documents the validation phase of CANvisionNative in full: the testing methodology, the repository audit, a full root-cause analysis of the alerts produced on a real vehicle recording, six confirmed software defects found and corrected during the engineering correction phase, the before/after measurements that quantify the effect of every correction, and an assessment of why the resulting Release Candidate is considered ready from a software point of view.

5.2 Validation methodology

The validation work followed one rule throughout: *every claim had to be backed by reproducible evidence before any code was changed*. Threshold tuning, blind retraining, or reducing the alert count by construction were explicitly excluded as acceptable fixes. A defect was only considered found once it could be triggered on demand and explained at the code level, and only considered fixed once the same reproduction steps, run again, produced the expected result.

This was applied in three passes, summarized in Table 5.1 and Figure 5.1.

Table 5.1: QA timeline.

Pass	Description
1. Interactive UI audit	Manual, jury-style exploratory testing of the running application: every screen, every control, wrong-order actions (importing twice, detecting before a model is loaded, spamming clicks), destructive actions, resizing and minimizing during a replay.
2. Root-cause analysis of alert volume	Full replay of a real one-hour Kia EV6 recording; every one of the resulting alerts explained and attributed to a dominant cause, until coverage reached 100%.
3. Correction and re-validation	Each confirmed software defect fixed individually, re-validated with direct evidence, then a full end-to-end re-run of the corrected pipeline (import, replay, detection, AI explanation, export) on the same real recording.

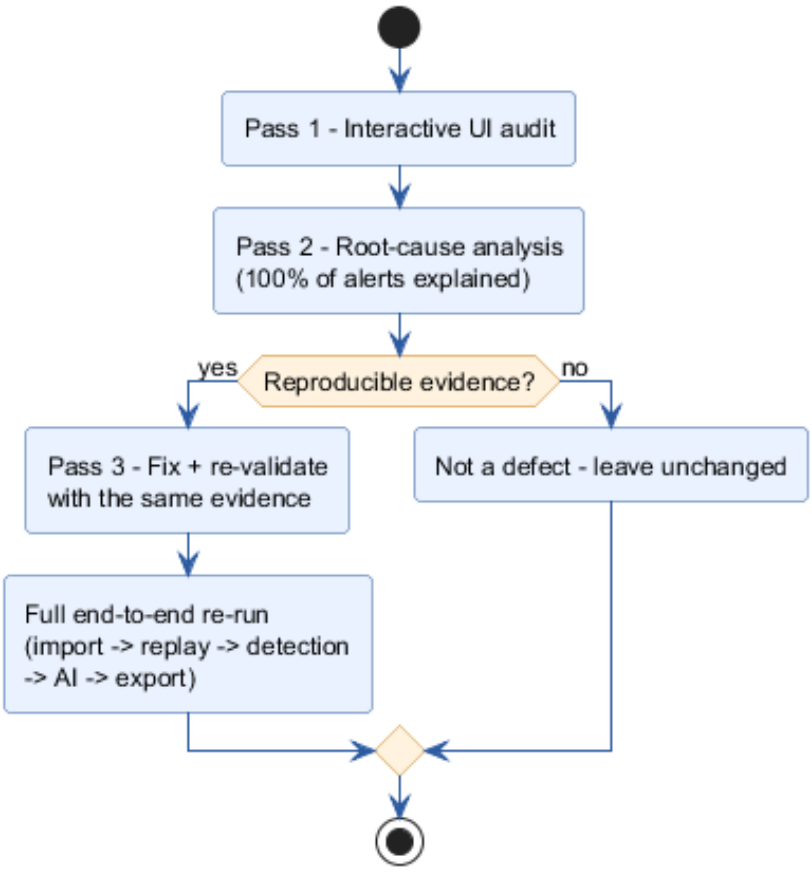

Figure 5.1: Validation workflow: three passes, gated by reproducible evidence.

Table 5.2 lists what was tested at each pass, the method used, and what it covered, to make the scope of the validation phase explicit.

Table 5.2: Testing matrix: what was covered, and how.

Target	Method	Coverage
Desktop UI (all screens)	Interactive UI automation (button clicks, file dialogs, keyboard input)	Navigation, import, replay controls, wrong-order and repeated actions
Detection pipeline	Direct backend API calls + full replay on real data	Vehicle selection, state classification, fusion scoring, reason attribution
Parsers	Synthetic fixtures per format (CSV, candump, Vector CAN) + real recordings	Canonical frame extraction, byte-for-byte comparison against known-good output
AI explanation	Direct API calls across two independent real datasets	Per-alert correctness, cross-session/cross-dataset leakage
Export	Manual export + direct file inspection	CSV column correctness, PDF rendering and pagination
End-to-end	Full application walk-through (import → replay → detection → AI → export)	Integration of every layer above, on the real Kia EV6 recording

5.2.1 Interactive UI audit

Before the data-driven root-cause analysis, the application was exercised interactively, screen by screen, using UI automation to drive the running desktop client (button clicks, file dialogs, keyboard input) while observing the result on screen. This pass covered the Home, Dashboard, Telemetry, and Diagnostics screens, log import with valid, empty, corrupted, and oversized files, the full replay lifecycle (start, pause, resume, stop, replay again), and deliberately wrong-order or repeated actions. It surfaced issues that were fixed before the data-driven validation pass, including an implausible-latency display appearing on three screens, a font glyph-substitution rendering bug, and destructive recovery actions that had no confirmation dialog.

5.2.2 Repository audit

Before changing any code, the repository was audited against its own internal documentation (`ROADMAP.md`, `SETUP.md`, `QA_PLAN.md`) to build an accurate map of every module before touching it, following the same principle applied throughout: understand before modifying, and never fix what is not demonstrably broken.

5.3 Real-data validation setup

5.3.1 Dataset

The main validation dataset is a real, one-hour Kia EV6 recording (`00000158-64CBC290.MF4`) from the CSS Electronics EV Data Pack, captured by a CANedge logger performing active UDS polling of the vehicle's ECUs rather than passively recording broadcast traffic. After MF4 conversion, the recording contains **97,741 frames**. Appendix C explains in detail why this specific recording became the official validation dataset. A second, independent dataset — a real Renault Clio Denial of Service (DoS)-attack candump log, from a different vehicle and a different logger format — was used specifically to test for cross-dataset state leakage (Section 5.5.6).

5.3.2 MF4 validation workflow

Figure 5.2 shows the workflow used to validate the MF4 conversion path before it was trusted for detection validation.

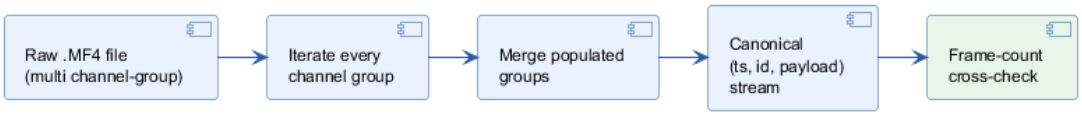

Figure 5.2: MF4 validation workflow: explicit per-group iteration followed by a frame-count cross-check.

The initial, naive conversion (a single call to the library's default dataframe view) only surfaced the first channel group and produced a small fraction of the real traffic. Iterating every group explicitly and cross-checking the resulting frame count against the file's own group metadata confirmed the fix and is what produced the 97,741-frame figure used for every later measurement in this chapter.

5.4 Root-cause analysis of the initial alert volume

Replaying the 97,741-frame recording through the pipeline, before any correction, produced **15,945 alerts** — an alert rate of 16.3%, high enough to question whether the system was usable. Rather than lowering the alert threshold, every alert was categorized by its dominant contributing layer, until 100% of the alerts were explained. Table 5.3 shows the result.

Table 5.3: Root-cause breakdown of the 15,945 baseline alerts.

Dominant cause	**Share**	**Explanation**
Machine-learning layer	61.42%	Wrong model selected + state misclassification (Fixes 1 and 2)
Timing layer	31.51%	Sparse UDS polling intervals, structurally ∼50–100% per-identifier alert rate
Payload layer	7.07%	Genuine sensor-value entropy in a real battery-telemetry message
Total explained	**100%**	

This breakdown is what directed the correction effort: the ML-dominant majority pointed directly at the model-selection and state-classification defects described next, while the timing- and payload-dominant shares were found to be structurally explainable by the UDS polling pattern of the recording and genuine sensor entropy, and were therefore not treated as defects.

5.5 Engineering correction phase

Four defects were found directly from the root-cause analysis above. Two further defects were found while re-validating the fixes end to end in the live application, rather than through the backend API alone. Each is presented below using the same structure: Problem, Investigation, Root cause, Fix, Evidence, and Result.

5.5.1 Fix 1 — Vehicle model selection

Problem. Selecting a vehicle profile in the client had no visible effect on detection: every replay appeared to score frames the same way regardless of which vehicle was selected.

Investigation. The vehicle identifier was traced from the UI selector, through the REST call, into the replay subprocess, and finally into the scoring call itself.

Root cause. `RealtimeEngine.process_frame()` never passed the selected vehicle identifier to `RuntimeModelStore.score()`. As a result, every replay was scored with the generic, non-vehicle-specific model, regardless of which vehicle profile the analyst selected.

Fix. The vehicle identifier was threaded through the engine's constructor and every call site that creates it: the replay engine, the offline analyzer, the live hardware interface (including a reset to the generic profile on disconnect), the REST endpoint, and the full C# call chain down to the view model bound to the vehicle selector.

Evidence. The corrected subprocess command line was captured directly from the operating system's process list and confirmed to include `–vehicle kia_ev6`; the `/api/vehicles` endpoint was confirmed to return an identifier (`kia_ev6`) that matches the on-disk trained-model directory exactly, closing the loop from the UI selector to the model actually loaded.

Result. Every replay now scores frames with the model bundle matching the selected vehicle profile, closing the gap between FR2 and the actual detection behaviour.

5.5.2 Fix 2 — Driving-state classification

Problem. The state classifier reported **99.93%** of the recording (97,677 of 97,741 frames) as "parking", despite the recording being an active one-hour drive.

Investigation. The feature vector and the scaling call feeding `classify_state()` were compared, field by field, against the feature list and scaler actually used when the K-Means model was trained.

Root cause. Two compounding defects were found. First, the feature vector passed to `kmeans.predict` was built from an 11-feature list intended for the scoring model, not the 14-feature list the K-Means model was actually trained on, which raised `ValueError: X has 11 features, but KMeans is expecting 14 features` on every single call. Second, this exception was silently caught by a broad `except` clause that always returned "parking", making the failure deterministic rather than intermittent, and therefore invisible without deliberately reproducing the exact exception.

Fix. The feature list used for state classification was corrected to the full 14-feature order the K-Means model was trained on, and the missing `RobustScaler.transform()` call was added before `kmeans.predict()`, matching the training-time scaling exactly.

Evidence. After the fix, the same recording classified as **99.93% "driving"** (97,667 of 97,741 frames), 64 "unknown", and 10 "parking" — the inverse distribution, and a credible one for an hour-long drive.

Result. The correct per-state Isolation Forest is now selected for almost every frame instead of the wrong, permanently-parking one, which is the single largest contributor to the alert-volume and ML-saturation improvements reported in Section 5.6.

5.5.3 Fix 3 — Reason attribution

Problem. The label shown to the analyst as the "dominant detection layer" of an alert did not match what the fusion score actually computed.

Investigation. The code path producing the displayed reason label was traced back from the API response to the internal reason string, and compared against `ScoringEngine.score()`'s actual inputs.

Root cause. The label was computed by taking the maximum of every `key=value` pair in the alert's internal reason string, including two diagnostic-only fields — `freq` (a raw frequency in Hz, sometimes in the thousands) and `can_id_entropy` (a raw entropy value) — that are never part of the actual fusion score. Because these two fields are numerically far larger than the properly normalized 0–2 range of the four real fusion inputs, they almost always won the comparison, mislabeling the displayed cause of the alert. The same defect, written independently, was found duplicated across four files (six occurrences in total): three in the main API module, and one each in the live hardware interface, the live simulator, and the statistics module.

Fix. All six occurrences were corrected to restrict the comparison to the four fields that genuinely feed the fusion score (timing, payload, machine learning, persistence).

Evidence. `ScoringEngine.score()` was confirmed, by inspecting its signature, to accept only the three normalized layer scores as input (persistence is derived internally from a rolling window), proving the mislabeled field never affected real detection, only the displayed explanation. After the fix, the live `/api/replay/alerts` endpoint was confirmed to return correctly attributed labels (for example `continuity_break` when the payload layer genuinely dominates).

Result. The displayed root-cause label now matches the fusion score's real inputs everywhere in the codebase, which directly improves the trustworthiness of both the analyst-facing alert list and the AI explanation built from it.

5.5.4 Fix 4 — Replay parser consistency

Problem. Some log formats that the file picker explicitly allowed produced empty or nonsensical replay results.

Investigation. The replay engine's loading code was compared against the shared, format-aware loader described in Section 4.5.2, which the offline batch analyzer already used successfully for the same formats.

Root cause. The replay engine had its own, separate loading code that called a generic CSV reader directly, bypassing the shared, format-aware parser. Any `.log`, `.asc`, or `.txt` file selected through the same file picker used for CSV files was silently mis-parsed as malformed CSV.

Fix. The replay engine was changed to call the same shared, format-aware loader already used by the offline batch analyzer, removing the duplicated CSV-only code path entirely.

Evidence. The corrected CSV path was confirmed to produce byte-identical output to the previous code on all 97,741 frames; synthetic candump and Vector CAN fixtures, which previously produced all-zero timestamps and identifiers, were confirmed to parse their identifier, timestamp, and payload correctly after the fix.

Result. Replay and offline analysis now share one single parsing code path for every supported format, satisfying NFR4.

5.5.5 Fix 5 — Alert-polling timer left stopped

Problem. After importing a log from the Home screen, no later replay could deliver alerts to the Anomaly Intel view for the rest of the application's lifetime, not even a fresh, correctly configured one.

Investigation. This defect only appeared through the live desktop client, not through direct backend API calls, which is why it was missed by the backend-only tests behind Fixes 1–4 and only surfaced during the end-to-end re-validation pass.

Root cause. Importing a log from the Home screen calls a service method that stops the client's background polling timer before running its own analysis, and no code path restarted it

afterwards.

Fix. The polling timer is now (re)started every time a replay is triggered, regardless of which screen initiated it.

Evidence. A replay started after a Home-screen import was confirmed to deliver alerts to the Anomaly Intel view again, matching the behaviour of a replay started without a prior import.

Result. The alert-delivery path is now reliable regardless of the sequence of screens the analyst uses, closing a defect that a purely backend-level test could not have caught.

5.5.6 Fix 6 — AI explanation cache staleness

Problem. An explanation request for the alert the client was actually displaying could silently return text describing a different, unrelated alert.

Investigation. Two datasets from different vehicles (the Kia EV6 recording and the independent Renault Clio recording) were replayed one after the other, and the explanation returned for the same alert index was compared across both runs.

Root cause. The AI explanation endpoint resolves an alert index against a shared, process-lifetime list that, before this fix, was only ever populated by two other endpoints (offline batch analysis and attack injection) — never by a completed replay. This meant an explanation request could silently resolve against unrelated data left over from a previous analysis or injection run, or, during the short window before a replay finished, against whatever an even earlier session had left behind.

Fix. Two changes closed this. On replay completion, the replay's own alerts are now published into the same shared state already used by the other two endpoints, mirroring a pattern one of them already used. On every replay *start*, that shared state is cleared immediately, so no previous session's data can be resolved, even during the short window before the new replay completes.

Evidence. The explanation for every checked alert index was confirmed to match that alert's own identifier and score in both datasets, with no trace of the other dataset's data. A second, more targeted test seeded the shared state with the Renault Clio data, started the Kia EV6 replay, and queried the explanation endpoint *while the replay was still running, before completion*: it correctly returned an empty result instead of the seeded stale data, confirming the start-of-replay clearing closes the exact timing window the completion-only fix could not.

Result. An explanation now always corresponds to the alert currently on screen, in both the completed-replay and mid-replay cases, satisfying the trust requirement behind NFR3.

5.6 Before/after measurements

Table 5.4 summarizes every measurement taken on the same 97,741-frame Kia EV6 recording, before any correction and after all four data-driven fixes.

Table 5.4: Before/after comparison on the real Kia EV6 recording (97,741 frames).

Metric	**Before**	**After**
Total alerts	15,945 (16.3%)	11,971 (12.25%)
Replay duration	685 s	341.2 s
Throughput	∼143 fps	∼286 fps
`vehicle_state` distribution	99.93% parking	99.93% driving
ML-score saturation ($\geq$0.999)	77.6–78.6%	0.0%

Figure 5.3, Figure 5.4, and Figure 5.5 present the three headline measurements as bar charts, generated directly from the replay summary files produced by the corrected pipeline and from the measurements recorded during the correction phase.

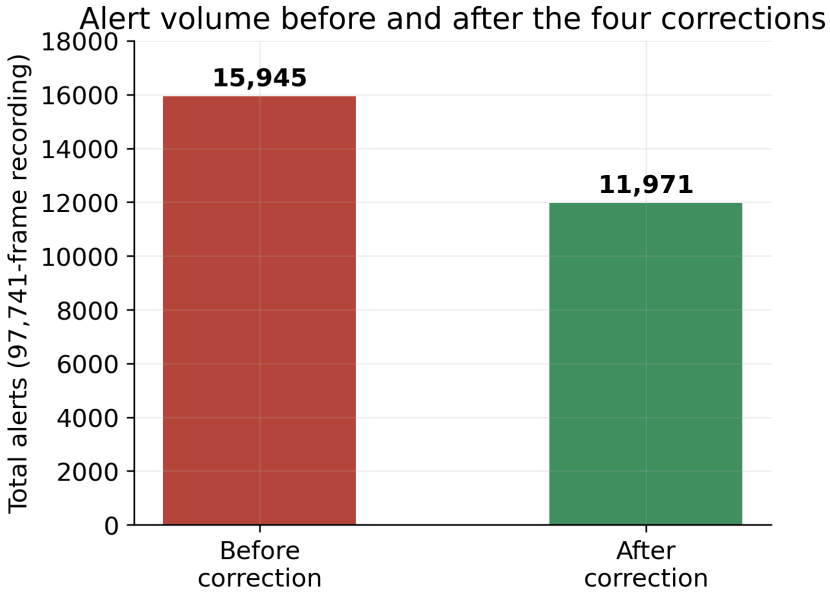

Figure 5.3: Total alerts on the Kia EV6 recording, before and after correction.

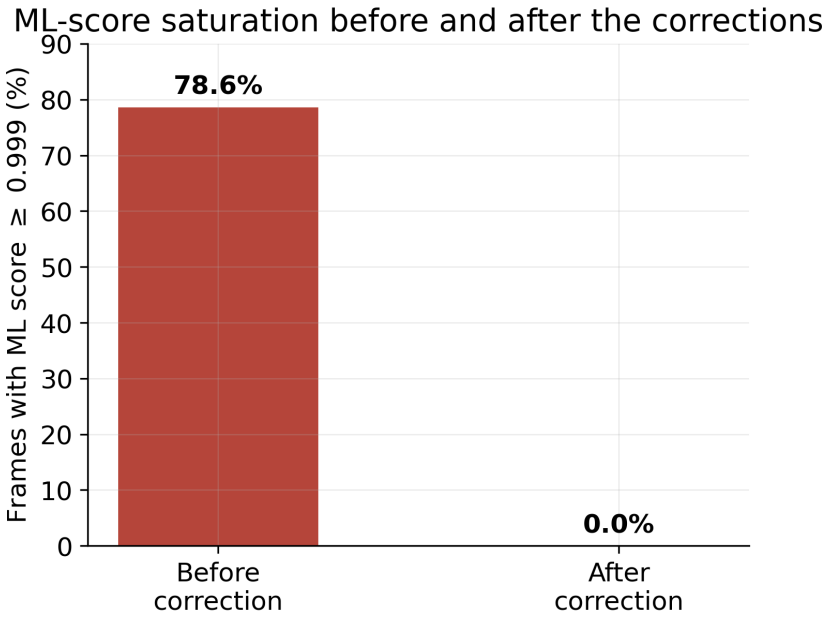

Figure 5.4: Share of frames with a saturated ($\geq$0.999) ML score, before and after correction.

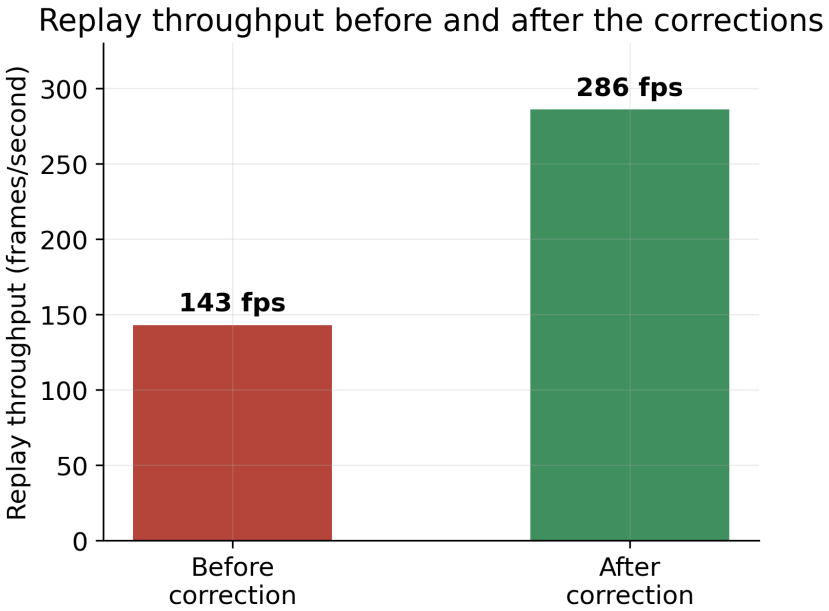

Figure 5.5: Replay throughput, before and after correction.

The alert reduction and the throughput improvement are both side effects of fixing genuine defects (the correct, lighter-weight per-vehicle model being exercised instead of a saturated fallback path), not of loosening the detection threshold, which was left unchanged throughout.

Figure 5.6 isolates the Fix 2 evidence already summarized in Section 5.5.2: the same 97,741 frames, classified by the state classifier before and after the correction, using the same three state labels on both sides so the two distributions can be compared directly.

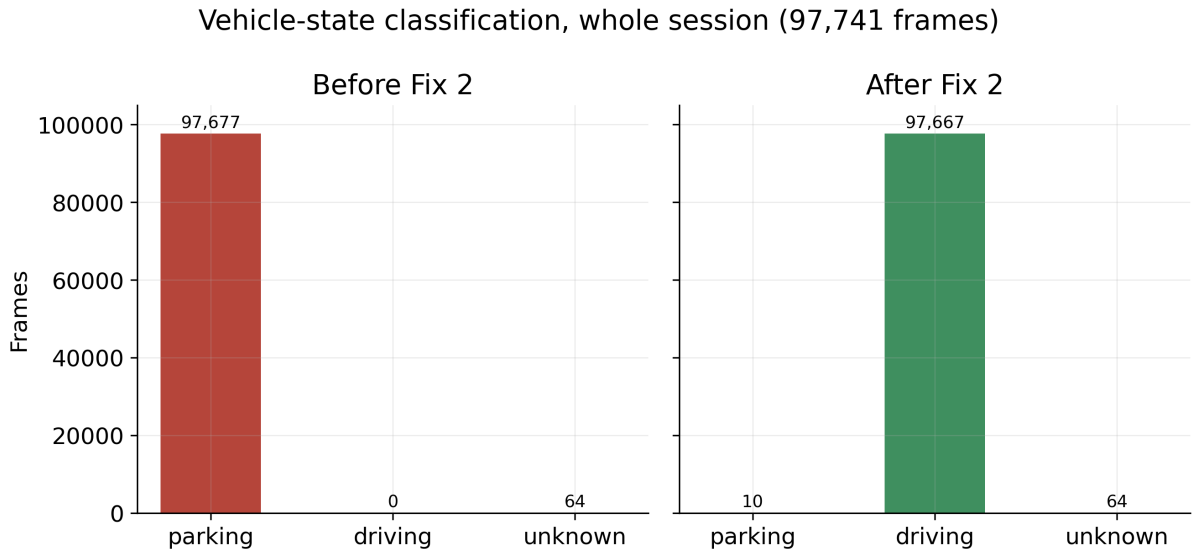

Figure 5.6: Vehicle-state classification of the whole recording, before and after Fix 2.

Figure 5.7 and Figure 5.8 break down the 11,971 post-correction alerts by severity and by dominant detection layer. The reason-distribution chart is itself evidence for Fix 3: none of the three buckets is `can_id_entropy` or `freq`, because those two fields were removed from the comparison entirely and were never part of the real fusion score in the first place (Section 3.7).

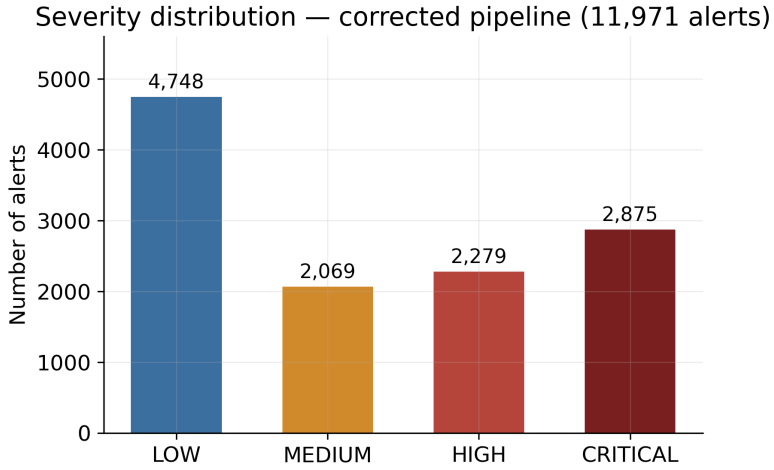

Figure 5.7: Severity distribution of the 11,971 alerts produced by the corrected pipeline.

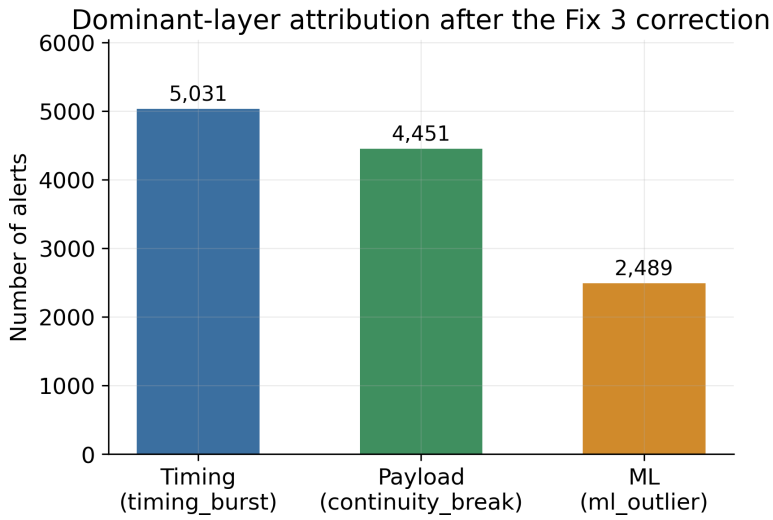

Figure 5.8: Dominant-layer attribution of the same 11,971 alerts, after the Fix 3 correction.

Figure 5.9 shows the fusion-score distribution of the first 1,000 alerts recorded by the corrected pipeline, against the three severity thresholds from Appendix B. The distribution is multi-modal rather than a single spike, and no bar sits at the top of the 0–2 range used internally by the fusion score: this is the direct, visual counterpart of the 0% ML-score-saturation figure in Table 5.4 — before the correction, the great majority of scores were pinned at the top of this range instead of spreading across it.

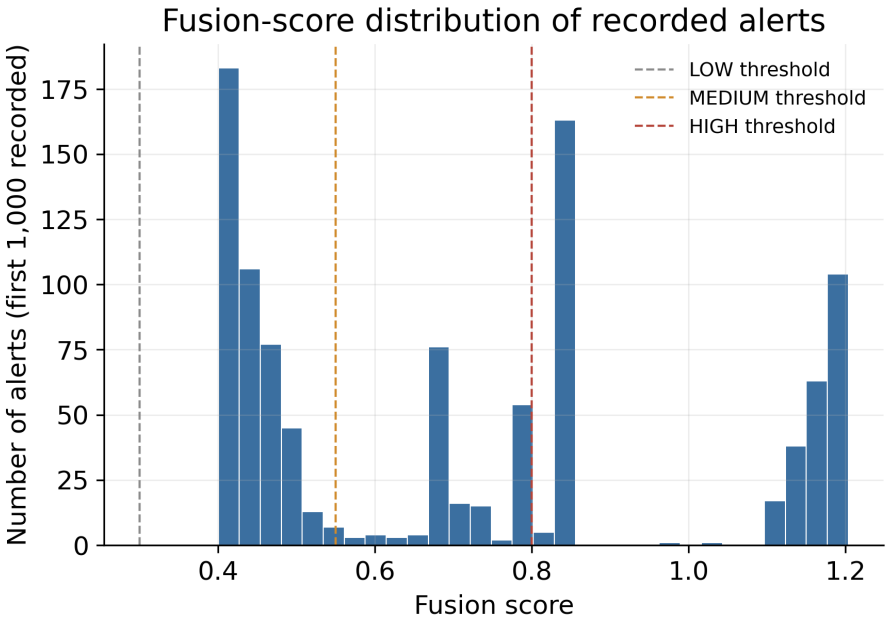

Figure 5.9: Fusion-score distribution of the first 1,000 alerts recorded by the corrected pipeline, against the LOW/MEDIUM/HIGH severity thresholds.

Figure 4.4 (Chapter 4) already shows this live alert list and a freshly-generated explanation for the selected alert, captured on the same 97,741-frame recording used throughout this section: the selected alert's identifier, severity, and score in the context bar (`0x7EC`, LOW, 0.415) match

exactly the summary text produced by the explanation engine, with no trace of an unrelated alert.

5.7 Export validation

CSV export was verified by inspecting the exported file directly: every row's time, identifier, severity, score, and attack-type columns matched the corresponding on-screen alert, and the parsed reason column reflected the Fix 3 correction. PDF export was verified by generating a report from the live application and confirming the file opens correctly and renders the same summary statistics and alert table as the CSV export.

5.8 Summary of engineering fixes

Table 5.5 summarizes every confirmed defect addressed during this project and its verification status.

Table 5.5: Summary of engineering fixes and their validation status.

#	Defect	Status
1	Vehicle model never selected for scoring	Fixed, verified
2	Driving-state misclassified as "parking"	Fixed, verified
3	Raw diagnostic fields dominating the displayed alert cause (6 occurrences)	Fixed, verified
4	Non-CSV replay formats bypassing the canonical parser	Fixed, verified
5	Alert-polling timer left permanently stopped after an import	Fixed, verified
6	AI explanation cache serving stale, cross-session data	Fixed, verified (two-part fix)

Figure 5.10 places the six fixes on the engineering timeline that produced them: the first four came directly out of the root-cause analysis of Section 5.4, while the last two only surfaced once the corrected pipeline was re-validated end to end in the live application rather than through the backend API alone — the exact reason Pass 3 of the methodology (Table 5.1) ends with a full end-to-end re-run and not just a backend re-test.

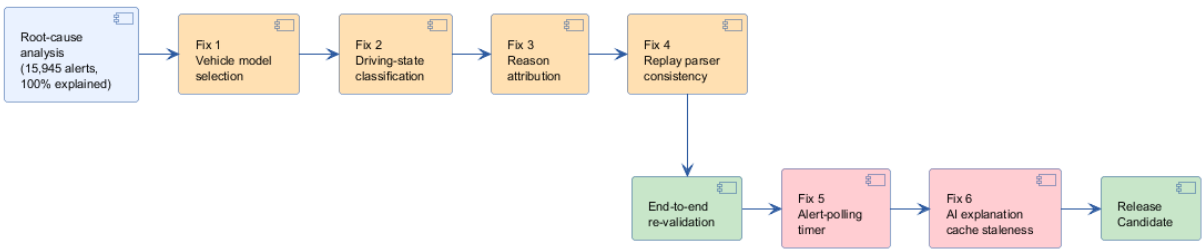

Figure 5.10: Bug-fix timeline: from root-cause analysis to the final Release Candidate.

5.9 Software readiness

This section states explicitly why the frozen repository state reached at the end of this project is considered a Release Candidate from a software point of view, and what that claim does and does not cover.

- **All identified defects are fixed and re-verified.** Every defect found during the root-cause analysis and the end-to-end re-validation (Table 5.5) was fixed and confirmed with direct evidence, not left as a known workaround.
- **Detection behaviour is now consistent across all three input modes.** Since Fix 1 and Fix 4, replay, live simulation, and live hardware capture all go through the same vehicle-aware scoring path and the same canonical parser, instead of behaving differently depending on how a frame reached the engine.
- **The full pipeline was re-validated end to end**, not only at the level of individual functions: import, replay, detection, AI explanation, and export were all exercised together on the real Kia EV6 recording after every fix, and again as a final full run before freezing the repository.
- **Performance is adequate for the target use case.** A one-hour, 97,741-frame recording now replays in about 341 seconds (∼286 fps), well within the "a few minutes, not hours" target of NFR1.
- **Failure of an external dependency degrades gracefully rather than breaking the application.** If the LLM provider chain is entirely unreachable, the rule-based fallback still produces a complete explanation (NFR6); if a vehicle has no trained model bundle, the engine falls back to a generic bundle and then to a lightweight heuristic rather than failing to score a frame at all.
- **What "ready" does not cover.** Readiness here is a software-level judgment, not a certification. The live hardware capture path is code-complete but has not been exercised against a physical vehicle (see Limitations below), and the CSV formatting defect described in Chapter 4's Export section remains open as a known, low-severity issue.

Taken together, these points are the basis for treating the frozen repository as production-ready *from a software engineering standpoint*: every known defect in the software itself has been fixed and verified, even though one input mode (physical hardware) still needs a real-vehicle test before it can be called validated in the same sense as replay and simulation.

5.10 Discussion

Every defect in Table 5.5 shares a common shape: the surrounding code did not crash and did not raise a visible error, so nothing pointed at the defect except a deliberate, evidence-based comparison between what the system reported and what was independently known to be true

(the vehicle actually being driven, the file actually being a valid log, the alert actually being the one on screen). This is the practical argument, beyond the specific numbers in Table 5.4, for validating an anomaly-detection system against real, independently-understood data rather than trusting that a correctly-structured pipeline behaves correctly end to end.

5.11 Limitations

The live hardware capture path (Chapter 3) remains code-complete but unvalidated against a physical vehicle, because the ELM327 adapter needed for that test did not arrive during the project. The AI explanation's "similar historical case" feature was found, during this validation phase, to intentionally persist across sessions by design (Chapter 3); this is documented here as a known, deliberate characteristic rather than a defect, but it means an analyst comparing two unrelated vehicles may see a cross-referenced historical case, which is worth keeping in mind when reading an explanation. The CSV exporter writes the `Score` column with a French-locale decimal comma without quoting it, which shifts every later column when the file is read by a plain comma-delimited parser (Appendix D); this is a minor formatting defect in the export path itself, distinct from the six defects listed in Table 5.5, and does not affect any detection or explanation result. Finally, the root-cause analysis in Section 5.4 was performed on one recording from one vehicle; the timing- and payload-dominant shares found structurally explainable there are not guaranteed to hold in the same proportions on a different vehicle or driving pattern.

5.12 Future improvements

Three of the limitations above point to concrete future work: validating the hardware capture path once an adapter is available, closing M2 from the project planning (Chapter 2); extending the root-cause methodology used in Section 5.4 to additional vehicles, to check whether the same timing/payload/ML split holds; and scoping the historical-case feature per vehicle or per session, if cross-vehicle references turn out to be more confusing than useful to analysts in practice. The CSV formatting defect is not included here, since its fix is a self-contained, well-understood change to the export code rather than an open engineering question.

5.13 Conclusion

This chapter presented the validation methodology followed, the real one-hour Kia EV6 recording used as the primary evidence base, the full root-cause analysis of its alert volume, the six confirmed defects it led to, each documented through its problem, investigation, root cause, fix, evidence, and result, the measured before/after impact of every correction, and the software-readiness assessment of the resulting Release Candidate. All measurements reported here are on the real recording described in Section 5.3, and every fix was verified with direct, reproducible evidence rather than by adjusting thresholds. The general conclusion that follows summarizes the project as a whole.

General Conclusion

This report presented CANvisionNative, a hybrid-architecture CAN diagnostic and validation platform for Battery Management Systems, from its motivation through its design, implementation, and validation.

Achievements

The project delivered a working desktop client and backend pipeline supporting three input modes: replay, live simulation, and live hardware capture, all through a shared code path. It delivered a four-layer detection pipeline (timing, payload, machine learning, temporal persistence) fused into a single score, and a vehicle-specific model store trained on more than seven million CAN frames across sixteen vehicle- and dataset-specific sources. It also delivered a natural-language explanation layer combining rule-based reasoning with a Large Language Model provider chain that degrades gracefully when no external model is reachable. Beyond the software itself, the project delivered a rigorous, evidence-based validation of that software against a real one-hour vehicle recording, documented in full in Chapter 5.

Objectives reached

Every objective listed in Chapter 1 was reached: the four-layer fusion pipeline was built and validated; recorded logs, live simulation, and live hardware capture are all supported through the same detection engine; vehicle-specific models are trained, selected at run time, and the selection was proven correct with direct evidence after a defect was found and fixed; the desktop client supports import, replay, review, and export; the explanation layer produces a plain-language description and recommendation for every alert; and the complete pipeline was validated end to end against real data, with every defect found fixed and re-verified rather than worked around.

Scientific contributions

Chapter 2 positioned this project against existing CAN tools and intrusion detection approaches on three points where most anomaly-based designs stop short: combining several independent detection signals into one fusion score instead of relying on a single signal, selecting a genuinely per-vehicle trained model at run time instead of one generic model, and producing a human-

readable explanation of a detected anomaly instead of a bare numeric score. The root-cause methodology used in Chapter 5 — explaining 100% of the alerts produced on a real recording by their dominant contributing layer before deciding whether any correction was needed — is itself a contribution that could be reused to audit other anomaly-detection systems built on the same fusion-score design.

Engineering contributions

Six confirmed software defects were found through evidence-based testing and corrected: incorrect vehicle-model selection, driving-state misclassification, mislabeled root-cause attribution (found duplicated across four files), a replay-parser inconsistency across log formats, a permanently-stopped alert-polling timer, and a stale-data leak in the AI explanation cache. Each was diagnosed to its exact root cause before being changed. Each fix was then verified with direct, reproducible evidence, measured on the same real recording before and after, rather than by tuning detection thresholds. Together, these fixes produced a measured 25% reduction in alert volume and eliminated machine-learning score saturation entirely, without artificially suppressing any alert.

Limitations

As detailed in Chapter 5, the live hardware capture path remains unvalidated on a physical vehicle, pending the arrival of a compatible adapter, and the root-cause proportions found on the Kia EV6 recording are not guaranteed to generalize to every vehicle without further testing.

Future work

The immediate next step is the hardware validation described in Chapter 2 as milestone M2, followed by extending the evidence-based root-cause methodology of Chapter 5 to additional vehicle profiles, and refining the scope of the historical-case reference feature discussed in Section 5.5.6.

Overall, this project shows that a desktop-class, multi-layer, explainable CAN diagnostic and validation platform for Battery Management Systems can be built with commodity hardware and open datasets. Just as importantly, it shows that such a platform can be held to the same standard of evidence as any other safety-relevant piece of software before it is trusted.

Bibliography

[1] International Organization for Standardization, *ISO 11898-1:2015 — road vehicles — controller area network (can) — part 1: data link layer and physical signalling*, ISO Standard, 2015.

[2] SAE International, *SAE J1979 — e/e diagnostic test modes*, SAE Standard, 2017.

[3] International Organization for Standardization, *ISO 14229-1:2020 — road vehicles — unified diagnostic services (uds) — part 1: application layer*, ISO Standard, 2020.

[4] F. Pedregosa et al., "Scikit-learn: machine learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest", in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, IEEE, 2008, pp. 413–422.

[6] J. MacQueen, "Some methods for classification and analysis of multivariate observations", *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, no. 14, pp. 281–297, 1967.

[7] ASAM e.V., *ASAM MDF — measurement data format, version 4*, ASAM Technical Standard, 2021.

[8] S. Ramírez, *FastAPI: modern, fast (high-performance) web framework for building APIs with Python*, https://fastapi.tiangolo.com, Accessed 2026, 2023.

[9] M. E. Verma, M. D. Iannacone, R. A. Bridges, S. C. Hollifield, B. Kay, and F. L. Combs, "Road: the real ornl automotive dynamometer controller area network intrusion detection dataset (with a comprehensive can ids dataset survey)", *arXiv preprint arXiv:2012.14600*, 2020.

[10] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network", in *Proceedings of the 16th Annual Conference on Privacy, Security and Trust (PST)*, IEEE, 2018.

Appendix A

REST API Endpoints

Table A.1 lists the REST endpoints exposed by `backend/api/server.py`, grouped by function. All endpoints are served on `http://127.0.0.1:8765` and consumed by the desktop client only.

Table A.1: Backend REST API endpoints.

Method	Path	Purpose
GET	`/health`	Backend health check
GET	`/vehicle`	Synthetic / live vehicle snapshot
GET	`/metrics`	Runtime metrics
GET	`/api/db/stats`	SQLite database statistics (frame count, DB size, recent alerts)
POST	`/analyze`	Offline batch analysis of one or more uploaded log files
GET	`/incidents`	Grouped incidents from the last analysis
POST	`/inject`	Attack injection into an uploaded log
POST	`/feedback`	Analyst feedback on a frame label
POST	`/infer`	One-off inference on posted features
POST	`/api/validation/score_file`	Batch validation scoring
POST	`/api/live/start`	Start live simulated session
POST	`/api/live/stop`	Stop live session
GET	`/api/live/signals`	Recent raw live frames
GET	`/api/live/decoded-signals`	Recent decoded live signals
GET	`/api/live/alerts`	Live alert buffer
GET	`/api/explain/{alert_index}`	AI explanation for an alert
GET	`/api/live/status`	Live/simulator status
GET	`/api/runtime/statistics`	Runtime IDS statistics
GET	`/api/runtime/health`	Runtime engine health
POST	`/api/replay/start`	Start a replay subprocess
POST	`/api/replay/stop`	Stop the running replay
GET	`/api/replay/status`	Replay progress / completion state
GET	`/api/replay/results`	Latest replay result file
GET	`/api/replay/alerts`	Alerts from the latest replay summary
GET	`/api/replay/partial-alerts`	Partial alerts while replay is running
POST	`/api/simulator/generate`	Generate a simulated attack
GET	`/api/simulator/attacks`	List available simulated attack types

Method	Path	Purpose
GET	`/api/validation/reports`	List validation report files
GET	`/api/vehicles`	Supported vehicle profile list
GET	`/api/vehicles/{vehicle_id}/can_ids`	CAN identifiers defined in a vehicle's DBC
POST	`/api/offline/analyze`	Start offline batch analysis
GET	`/api/offline/status/{session_id}`	Offline analysis progress
GET	`/api/offline/frames/{session_id}`	Paginated decoded frames
GET	`/api/offline/summary/{session_id}`	Offline analysis summary
POST	`/api/mf4/convert`	Convert an MF4 file to CSV
POST	`/api/live/hardware/connect`	Connect a physical CAN adapter
POST	`/api/live/hardware/disconnect`	Disconnect the adapter
GET	`/api/live/hardware/status`	Hardware connection status
GET	`/api/live/hardware/ports`	Available serial ports
GET	`/api/live/hardware/check`	Hardware availability check
GET	`/api/system/info`	System / build information
GET	`/api/settings`	Current settings
POST	`/api/settings`	Update settings
POST	`/api/chat`	Conversational Q&A about an alert
POST	`/api/session/record/start`	Start session recording
POST	`/api/session/record/stop`	Stop session recording
GET	`/api/session/record/status`	Recording status
GET	`/api/session/list`	List recorded sessions
GET	`/api/system/health`	Aggregated system health

Appendix B

Configuration

B.1 Requirements

Table B.1 lists the software requirements to build and run CANvisionNative, as documented in the project's own setup guide. Table B.2 lists the hardware requirements; only the last row is needed for the live hardware capture mode (Section 3.6) — every other feature, including the full validation in Chapter 5, runs on the desktop machine alone.

Table B.1: Software requirements.

Component	**Version**
Windows	10 or 11, 64-bit
.NET Runtime	4.8
Python	3.11 – 3.14
Git	Any (for cloning the repository)

Table B.2: Hardware requirements.

Component	**Requirement**	**Notes**
Desktop / laptop	Any machine able to run Windows 10/11 and Python 3.11+	Used for every mode (replay, simulation, hardware capture)
Disk space	A few GB free	Trained model store, DBC files, and recorded logs
USB port	1 free port	Only for the ELM327 / USB-CAN adapter below
CAN adapter	ELM327 USB (slcan), PEAK PCAN-USB, or a SocketCAN-compatible interface	Required only for live hardware capture (FR9); not needed for replay or simulation

B.2 Key Python dependencies

- `fastapi`, `uvicorn` — REST API server
- `scikit-learn`, `joblib` — Isolation Forest and K-Means pipeline
- `pandas`, `numpy` — feature engineering
- `asammdf` — MF4 decoding
- `cantools` — DBC-based signal decoding
- `python-can` — hardware CAN adapter access

B.3 Backend startup

```
python -m uvicorn backend.api.server:app --host 127.0.0.1 --port 8765
```

The desktop client automatically starts and stops this process when a Python interpreter is found at the expected path, and exposes the same functionality if the backend is instead started manually beforehand.

B.4 Persisted configuration

Client-side settings (alert threshold, LLM provider key, hardware interface parameters) are persisted to `%AppData%\CANvision\config.json` and reloaded on startup. The LLM provider key, if configured, is applied as an environment variable so the backend can pick it up without a separate configuration step.

B.5 Fusion score configuration

Table B.3: Default fusion and severity configuration.

Parameter	Value
Fusion score range	0.0 – 2.0
Layer weights	timing 0.35, payload 0.35, ML 0.20, persistence 0.10
Severity thresholds	LOW $\geq$ 0.30, MEDIUM $\geq$ 0.55, HIGH $\geq$ 0.80, CRITICAL $\geq$ 1.00
Backend port	8765

Appendix C

Datasets

This appendix describes every dataset used by CANvisionNative, either for training the machine-learning models or for validating the system, and explains why the Kia EV6 recording became the official validation dataset used throughout Chapter 5.

C.1 Training corpus

The machine-learning models used by CANvisionNative were trained on a corpus of **more than seven million CAN frames** drawn from sixty-six sources, summarized in Table C.1.

Table C.1: Main sources of the training corpus.

Source	Description	Frames
ROAD ambient [9]	Ambient (attack-free) drives from the ROAD dataset	1,737,195
Car-Hacking (normal rows) [10]	Public benchmark, normal-traffic subset	1,200,000
Nissan Leaf	Community EV CAN logs	1,163,181
BMW i3	Community EV CAN logs	1,044,993
Kia EV6	Community EV CAN logs	886,368
13 additional sources	Kia Soul, Jaguar I-PACE, MG ZS EV, Tesla, others	∼1,032,329

This corpus produced sixteen vehicle- or dataset-specific model folders (including `kia_ev6`, `nissan_leaf`, `bmw_i3`, `skoda_enyaq`, `tesla`, and others) plus one generic fallback bundle (`general`), stored under `assets/models/`, alongside the shared `kmeans_state_model.joblib` and `state_scaler.joblib` used by the state classifier (Section 4.6.5).

C.2 Dataset descriptions

Tables C.2 to C.5 describe each of the four dataset families used in the project — the primary validation recording, the two labelled academic benchmarks used for training, and the community-contributed EV logs used for training volume and diversity — against the same eight criteria: source, license, vehicle, format, purpose, trustworthiness, strengths, and limitations.

Tables C.2–C.5 give one table per dataset family, so each stays short enough to read as a single block.

Table C.2: CSS Electronics EV Data Pack (Kia EV6) — primary validation dataset.

Source	Published by CSS Electronics, a commercial manufacturer of CANedge data loggers, as a free public sample data pack demonstrating real-world CAN/UDS logging.
License / access	Freely downloadable from CSS Electronics' public data pack page for evaluation and demonstration purposes.
Vehicle	Kia EV6 (real, unmodified production vehicle).
Format	MF4 binary log (ASAM MDF v4), converted to a canonical CSV frame stream by this project's MF4 pipeline (Section 4.5.3).
Purpose in this project	Primary validation dataset for the whole of Chapter 5: root-cause analysis, all six engineering fixes, and every before/after measurement.
Trustworthiness	High: captured by a professional logger performing active UDS polling of a real vehicle's ECUs during an actual one-hour drive, giving an independently known ground truth against which the state-classification defect (Fix 2) could be directly checked.
Strengths	Real production vehicle; independently verifiable frame count against MF4 metadata; already has a named profile and trained model bundle; realistic UDS polling pattern.
Limitations	A single vehicle and a single one-hour session; does not by itself confirm that the same alert-rate proportions hold on other vehicles or driving styles (Chapter 5, Limitations).

Table C.3: ROAD dataset [9] — training corpus.

Source	Published by its authors (Verma et al.) as an open academic benchmark, described in the paper title as the "Real ORNL Automotive Dynamometer" CAN intrusion detection dataset.
License / access	Released publicly by the authors for research use.
Vehicle	Real vehicle traffic captured on a dynamometer test bench, with both ambient (attack-free) and labelled-attack recordings, including a distinction between plain injection and masquerade attacks.
Format	candump-style timestamped text log (`(ts) can0 ID#hexdata`).
Purpose in this project	Only the ambient (attack-free) recordings were used, as part of the training corpus, to expose the models to real, normal traffic patterns from a source independent of the community EV logs.
Trustworthiness	High as an academic benchmark purpose-built for IDS evaluation, though captured on a dynamometer bench rather than on open roads; used here only for its normal-traffic statistics.
Strengths	Peer-reviewed provenance; explicit ambient/attack separation makes it safe to use only the ambient part; adds a training source independent of the community EV logs.
Limitations	Dynamometer traffic, not open-road traffic; attack labels were not used in this project, so this dataset contributes only volume and diversity, not attack ground truth.

Table C.4: Car-Hacking dataset [10] — training corpus.

Source	Published by the Hacking and Countermeasure Research Lab (HCRL), used in the GIDS paper as a benchmark for GAN-based intrusion detection.
License / access	Publicly released by HCRL for research use.
Vehicle	Real vehicle CAN traffic with labelled attack and normal traffic.
Format	CSV, `timestamp, can_id, dlc, b0..b7` columns, plus a label column.
Purpose in this project	Only the normal-traffic rows (no attack labels) were used, as part of the training corpus, for the same reason as the ROAD ambient data: real, normal traffic diversity from a second independent public source.
Trustworthiness	High as one of the most widely cited public CAN IDS benchmarks; used here strictly for its normal rows.
Strengths	Well-known, widely cited benchmark, which makes its normal-traffic statistics easy to cross-check against other published work.
Limitations	Captured on a single test vehicle under controlled conditions; like ROAD, its attack labels were not used, only its normal rows.

Table C.5: Community EV CAN logs — training corpus.

Source	CAN logs shared publicly by EV enthusiast and vehicle-research communities, covering the Nissan Leaf, BMW i3, Kia EV6, Kia Soul, Tesla, Jaguar I-PACE, MG ZS EV, Hyundai Ioniq 5, Škoda Enyaq, and other platforms.
License / access	Shared informally and publicly by their contributors; used here for internal model training only, not redistributed as part of this project's deliverables.
Vehicle	One real vehicle per source, matching the vehicle profiles listed in Table 4.2.
Format	Mostly SavvyCAN-style or candump-style CSV logs.
Purpose in this project	Bulk of the training corpus, giving each supported vehicle profile its own trained Isolation Forest bundle instead of relying on one generic model.
Trustworthiness	Variable and less formally documented than the two academic benchmarks above; mitigated by using them only for training, not for any validation claim in Chapter 5.
Strengths	Wide vehicle coverage, which is exactly what makes the sixteen per-vehicle model bundles (Appendix C) possible instead of a single generic model.
Limitations	Informal provenance and inconsistent documentation quality between sources; not used for any validation claim in Chapter 5, only for training.

C.3 Selection criteria

Model training favoured breadth: as many independent vehicles and sources as could be gathered, so that the per-vehicle model store (Appendix C, Section 4.5.4) would not depend on a single manufacturer's traffic pattern. Validation had the opposite priority: depth and independent verifiability on one recording, rather than breadth across many. A dataset was suitable for validation only if it met four conditions: it had to come from a real, unmodified vehicle rather than a synthetic simulation; it had to be long enough to produce statistically meaningful before/after measurements; its frame count had to be independently cross-checkable against the raw file's own metadata, to rule out a parsing defect before it could be mistaken for a detection defect; and the vehicle had to already have a named profile and a trained model bundle, so that the full vehicle-aware detection path — not just the generic fallback — could be exercised end to end.

C.4 Why the Kia EV6 recording became the official validation dataset

The Kia EV6 recording from the CSS Electronics EV Data Pack was the only real-vehicle recording available that satisfied all four conditions above at once. It is a genuine, unmodified production vehicle, not a bench or simulated setup. At 97,741 frames over roughly one hour, it is long enough that a percentage-point change in alert rate or ML-score saturation (Table 5.4) reflects a real shift in behaviour rather than noise from a handful of frames. Its frame count could be independently cross-checked against the MF4 file's own channel-group metadata (Section 5.3), which is what first exposed the MF4 conversion defect described there, before any detection-level claim was trusted. Finally, `kia_ev6` is one of the nine named vehicle profiles in Table 4.2, with its own DBC file and trained model bundle, so replaying it exercises the full per-vehicle detection path — the exact path Fix 1 (Chapter 5) proved was previously broken. The active UDS polling pattern of the recording was an additional, deliberate advantage: it produces a structured, request/response traffic pattern rather than uniform background noise, which is what allowed the timing layer's contribution to the baseline alert volume (Table 5.3) to be explained structurally instead of dismissed as unexplained noise.

C.5 Cross-validation dataset

To test for cross-session and cross-dataset state leakage in the AI explanation module (Section 5.5.6), a second, independent real recording was used: a Renault Clio DoS-attack candump log, from a different vehicle and a different capture tool, containing frames on identifiers unrelated to the Kia EV6 recording (for example `0xC6`, `0x511`). This dataset was chosen specifically because it shares no vehicle profile, no logger format, and no identifier space with the Kia EV6 recording, which makes any cross-contamination between the two immediately obvious rather than masked by coincidental overlap.

Appendix D

Additional Validation Data

D.1 Post-correction alert breakdown

Table D.1 and Table D.2 give the full severity and reason-attribution breakdown of the 11,971 alerts produced by the corrected pipeline on the Kia EV6 recording (Chapter 5), superseding the pre-correction breakdown of Table 5.3.

Table D.1: Severity distribution after correction.

Severity	**Count**	**Share**
LOW	4,748	39.7%
MEDIUM	2,069	17.3%
HIGH	2,279	19.0%
CRITICAL	2,875	24.0%
Total	**11,971**	**100%**

Table D.2: Reason-attribution distribution after the Fix 3 correction.

Dominant layer (bucket)	**Count**	**Share**
Timing (`timing_burst`)	5,031	42.0%
Payload (`continuity_break`)	4,451	37.2%
ML (`ml_outlier`)	2,489	20.8%
Total	**11,971**	**100%**

D.2 Top alerting CAN identifiers

Figure D.1 ranks the eight most frequent identifiers in the post-correction alert list. The identifiers `0x7EC`, `0x7E4`, `0x7BB`, `0x7A8`, `0x7B3`, and `0x7A0` all fall in the `0x7A0–0x7EC` range reserved for UDS diagnostic request/response pairs, consistent with the recording's active-polling capture method described in Appendix C. Identifier `0x19`, the single most frequent alert source, and `0x13` are broadcast identifiers rather than diagnostic ones, and correspond to the high-frequency battery and drivetrain messages discussed in Section 5.4.

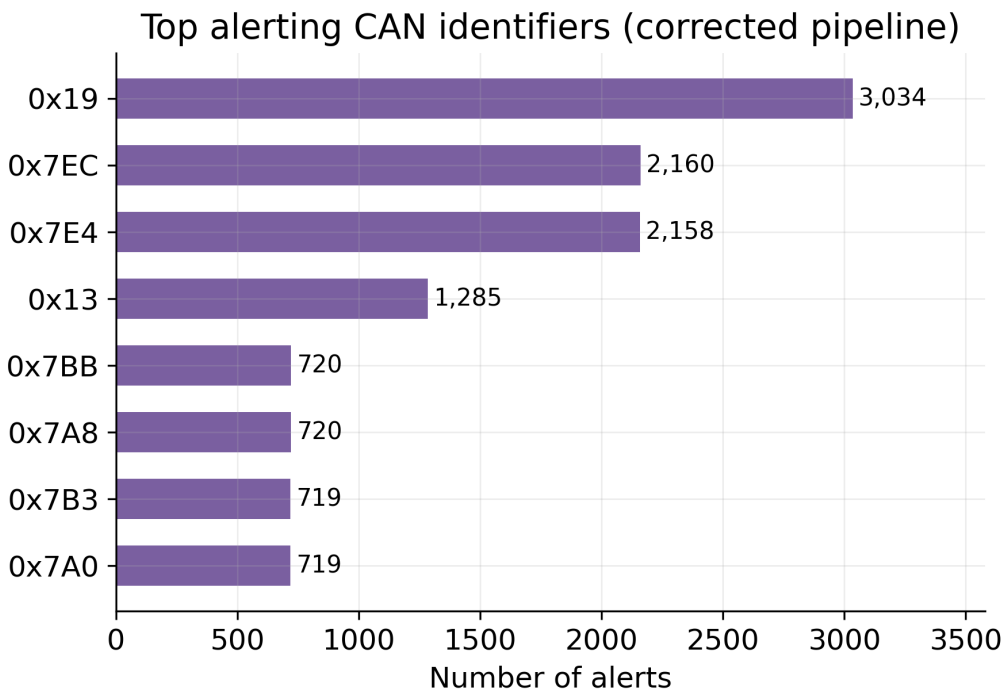

Figure D.1: Top alerting CAN identifiers in the corrected pipeline's alert list.

D.3 Additional screenshots

Figure D.2 shows the Dashboard view during a live session: the fusion score gauge, active-threat panel, and IDS quick statistics.

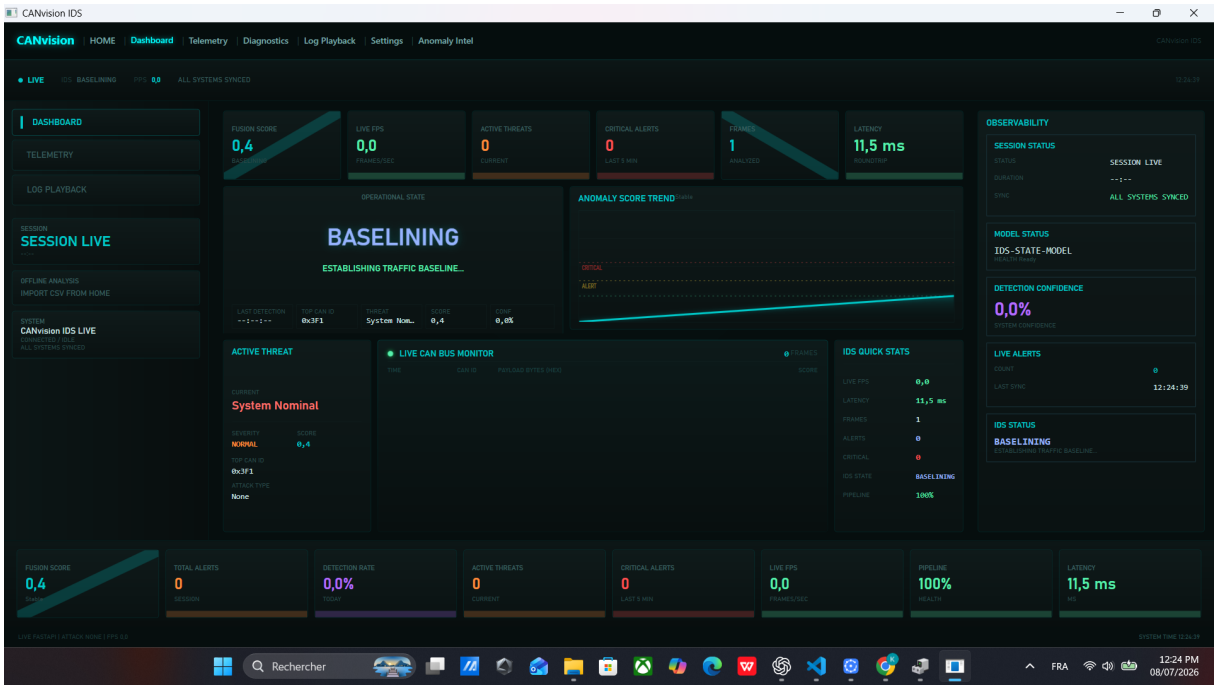

Figure D.2: Dashboard view: fusion score, active-threat panel, and IDS quick statistics.

Figure D.3 shows the Settings view, including the hardware interface fields, the alert threshold slider, and the vehicle profile list also summarized in Table 4.2.

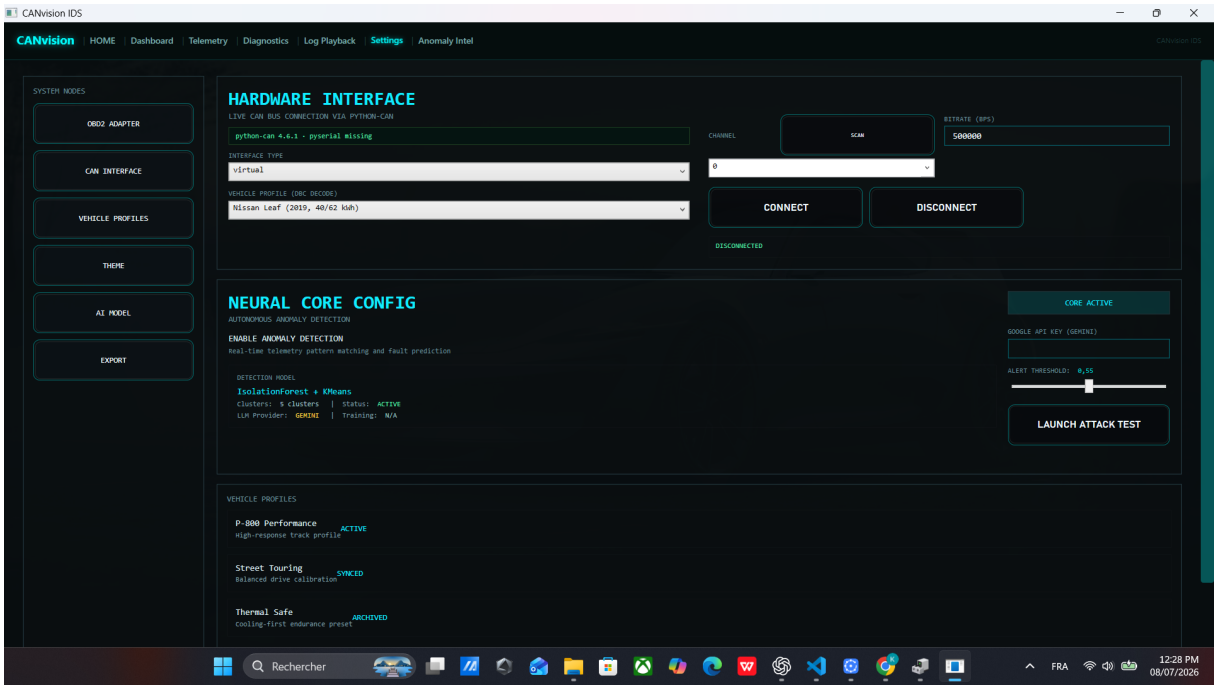

Figure D.3: Settings view: hardware interface, neural core configuration, and vehicle profiles.

Figure D.4 shows a rendering of the real exported CSV file, reconstructed from the raw export because the exporter writes the `Score` column with a French-locale decimal comma (for example `0,415`) without quoting it, which shifts every later column when the file is opened with a plain, comma-delimited reader. This is a genuine, minor formatting quirk of the export path, noted here for completeness; it does not affect any of the validation results in Chapter 5, all of which were read from the underlying JSON summary rather than the CSV export.

Exported alerts_20260708_125731.csv — first 12 rows (real export, corrected pipeline)

Time	CAN_ID	Severity	AttackType	Score
01:00:00.028	0x7EC	LOW	CONTINUITY_BREAK	0.415
01:00:00.517	0x19	LOW	CONTINUITY_BREAK	0.440
01:00:00.521	0x7BB	MEDIUM	CONTINUITY_BREAK	0.624
01:00:01.000	0x7E4	LOW	CONTINUITY_BREAK	0.493
01:00:01.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0.607
01:00:01.528	0x19	LOW	CONTINUITY_BREAK	0.426
01:00:01.530	0x7A8	LOW	CONTINUITY_BREAK	0.414
01:00:02.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0.636
01:00:02.028	0x7EC	LOW	CONTINUITY_BREAK	0.541
01:00:02.540	0x19	LOW	CONTINUITY_BREAK	0.428
01:00:04.573	0x19	LOW	CONTINUITY_BREAK	0.486
01:00:05.000	0x7E4	HIGH	TIMING_BURST	0.838

Figure D.4: Exported CSV file (reconstructed preview), Time/CAN_ID/Severity/AttackType/Score columns.

Figure D.5 shows the first page of the exported PDF report, generated from the same completed replay: header, summary statistics, and the start of the paginated alert table.

CANVISION IDS

INTRUSION DETECTION SYSTEM — ALERT REPORT

Generated: 2026-07-08 13:21:13

ANALYSIS SUMMARY

TOTAL ALERTS	CRITICAL	HIGH / WARNING	LOW
500	103	78	319

ALERT DETAILS

#	TIME	CAN-ID	SEVERITY	ATTACK TYPE	SCORE	REASON
1	01:00:00.028	0x7EC	LOW	CONTINUITY_BREAK	0,415	timing=0.000, payload=1.185, ml=0.000, persistence=0.000, freq=36Hz, can_id_entropy=0.000
2	01:00:00.517	0x19	LOW	CONTINUITY_BREAK	0,440	timing=0.004, payload=0.915, ml=0.591, persistence=0.000, freq=10Hz, can_id_entropy=1.535
3	01:00:00.521	0x7BB	MEDIUM	CONTINUITY_BREAK	0,624	timing=0.014, payload=1.769, ml=0.000, persistence=0.000, freq=96Hz, can_id_entropy=1.535
4	01:00:01.000	0x7E4	LOW	CONTINUITY_BREAK	0,493	timing=0.470, payload=0.938, ml=0.000, persistence=0.000, freq=2Hz, can_id_entropy=1.535
5	01:00:01.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,607	timing=0.266, payload=1.045, ml=0.740, persistence=0.000, freq=10Hz, can_id_entropy=2.163
6	01:00:01.528	0x19	LOW	CONTINUITY_BREAK	0,426	timing=0.004, payload=0.794, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.262
7	01:00:01.530	0x7A8	LOW	CONTINUITY_BREAK	0,414	timing=0.000, payload=1.183, ml=0.000, persistence=0.000, freq=42Hz, can_id_entropy=2.262
8	01:00:02.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0,636	timing=0.470, payload=0.924, ml=0.740, persistence=0.000, freq=2Hz, can_id_entropy=2.262
9	01:00:02.028	0x7EC	LOW	CONTINUITY_BREAK	0,541	timing=0.289, payload=0.836, ml=0.739, persistence=0.000, freq=8Hz, can_id_entropy=2.262
10	01:00:02.540	0x19	LOW	CONTINUITY_BREAK	0,428	timing=0.003, payload=0.801, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.281
11	01:00:04.573	0x19	LOW	CONTINUITY_BREAK	0,486	timing=0.003, payload=0.967, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.267
12	01:00:05.000	0x7E4	HIGH	TIMING_BURST	0,838	timing=1.044, payload=0.925, ml=0.740, persistence=0.010, freq=1Hz, can_id_entropy=2.267
13	01:00:05.001	0x7EC	MEDIUM	CONTINUITY_BREAK	0,753	timing=0.644, payload=1.080, ml=0.742, persistence=0.010, freq=4Hz, can_id_entropy=2.267
14	01:00:05.481	0x13	LOW	ML_OUTLIER	0,478	timing=0.433, payload=0.505, ml=0.737, persistence=0.020, freq=1Hz, can_id_entropy=2.151
15	01:00:05.482	0x15	LOW	ML_OUTLIER	0,457	timing=0.433, payload=0.450, ml=0.732, persistence=0.020, freq=1Hz, can_id_entropy=2.151
16	01:00:05.490	0x7BB	CRITICAL	TIMING_BURST	1,194	timing=2.000, payload=0.981, ml=0.740, persistence=0.030, freq=1Hz, can_id_entropy=2.151
17	01:00:05.500	0x7B3	HIGH	TIMING_BURST	0,978	timing=2.000, payload=0.784, ml=0.000, persistence=0.040, freq=0Hz, can_id_entropy=2.151
18	01:00:05.689	0x19	MEDIUM	CONTINUITY_BREAK	0,557	timing=0.003, payload=1.141, ml=0.733, persistence=0.100, freq=10Hz, can_id_entropy=2.235
19	01:00:06.000	0x5	LOW	ML_OUTLIER	0,404	timing=0.400, payload=0.312, ml=0.727, persistence=0.100, freq=1Hz, can_id_entropy=2.235
20	01:00:06.000	0x7E4	HIGH	TIMING_BURST	0,818	timing=0.945, payload=0.938, ml=0.740, persistence=0.110, freq=1Hz, can_id_entropy=2.235
21	01:00:06.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,720	timing=0.557, payload=1.045, ml=0.742, persistence=0.110, freq=5Hz, can_id_entropy=2.235
22	01:00:06.483	0x13	LOW	ML_OUTLIER	0,484	timing=0.400, payload=0.531, ml=0.737, persistence=0.110, freq=1Hz, can_id_entropy=2.175
23	01:00:06.500	0x7A0	CRITICAL	TIMING_BURST	1,017	timing=2.000, payload=0.873, ml=0.000, persistence=0.120, freq=0Hz, can_id_entropy=2.175
24	01:00:06.504	0x7A8	CRITICAL	TIMING_BURST	1,171	timing=1.836, payload=1.048, ml=0.742, persistence=0.130, freq=1Hz, can_id_entropy=2.175
25	01:00:07.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0,797	timing=0.877, payload=0.924, ml=0.737, persistence=0.190, freq=1Hz, can_id_entropy=2.254
26	01:00:07.000	0x5	LOW	ML_OUTLIER	0,404	timing=0.373, payload=0.312, ml=0.727, persistence=0.190, freq=1Hz, can_id_entropy=2.254
27	01:00:07.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,643	timing=0.523, payload=0.836, ml=0.740, persistence=0.190, freq=6Hz, can_id_entropy=2.254
28	01:00:07.115	0x19	LOW	CONTINUITY_BREAK	0,522	timing=0.003, payload=1.016, ml=0.733, persistence=0.190, freq=10Hz, can_id_entropy=2.254
29	01:00:07.496	0x13	LOW	ML_OUTLIER	0,466	timing=0.371, payload=0.485, ml=0.737, persistence=0.190, freq=1Hz, can_id_entropy=2.254
30	01:00:08.230	0x19	LOW	ML_OUTLIER	0,413	timing=0.003, payload=0.705, ml=0.733, persistence=0.180, freq=10Hz, can_id_entropy=2.268
31	01:00:09.000	0x5	LOW	ML_OUTLIER	0,409	timing=0.331, payload=0.375, ml=0.727, persistence=0.170, freq=1Hz, can_id_entropy=2.227
32	01:00:09.356	0x19	LOW	CONTINUITY_BREAK	0,439	timing=0.003, payload=0.785, ml=0.733, persistence=0.170, freq=10Hz, can_id_entropy=2.227
33	01:00:09.480	0x13	LOW	ML_OUTLIER	0,476	timing=0.330, payload=0.567, ml=0.737, persistence=0.140, freq=1Hz, can_id_entropy=2.227

CANvision Native — CAN Bus Intrusion Detection System

Page 1

Figure D.5: Exported PDF report, first page.

Appendix E

Data Sources Summary

This appendix gives a one-page summary of every dataset used in this project and where it came from. Appendix C gives the full detail (license, format, purpose, trustworthiness, strengths, and limitations) for each one.

Table E.1: Every dataset used, and its source.

Dataset	Vehicle / origin	Source
CSS Electronics EV Data Pack (primary validation dataset)	Kia EV6 (real production vehicle)	Published by CSS Electronics, a commercial manufacturer of CANedge CAN data loggers, as a free public sample data pack.
ROAD dataset [9]	Real vehicle, dynamometer test bench	Released publicly by its authors for CAN intrusion-detection research.
Car-Hacking dataset [10]	Real vehicle	Released publicly by the Hacking and Countermeasure Research Lab (HCRL).
Community EV CAN logs	Nissan Leaf, BMW i3, Kia EV6, Kia Soul, Tesla, Jaguar I-PACE, MG ZS EV, Hyundai Ioniq 5, Škoda Enyaq, and other platforms	Shared publicly by EV enthusiast and vehicle-research communities.
Renault Clio DoS-attack log (cross-validation dataset)	Renault Clio (real vehicle)	Independent real candump recording, used only to test for cross-dataset state leakage (Section 5.5.6).

No dataset used in this project was purchased, licensed under a restrictive commercial agreement, or obtained under a non-disclosure arrangement. Every source listed above is either a public academic release, a vendor's own public sample data, or traffic shared openly by the EV research community.

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454